14 haftalık yol haritasında 3. hafta

ALGORİTMA VE PROGRAMLAMA – II · DERS NOTU

HAFTA 03

Özyineleme ile Arama Uzayını Dolaşmak

Üretme Algoritmaları ve Geri İzleme

MODÜL I · ALGORİTMALARIN DİLİ · H1–H3

HAFTANIN BÜYÜK SORUSU

Satranç tahtasına birbirini tehdit etmeyen sekiz vezir kaç farklı biçimde yerleştirilebilir ve yaklaşık 4,4 milyar yerleşimi tek tek denemeden yanıtı nasıl buluruz?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Bahar yarıyılı · Sürüm 1.0 · Ekim 2026
`Ortam: gcc -std=c17 · g++ -std=c++17 (-Wall -Wextra -pedantic -O2) · Python 3.10+`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları.... 4
1.1 Öğrenme çıktıları.... 4
2. Geçen Haftadan ve AP-1'den Bağlantı.... 5
3. Kavramsal Anlatım: Özyineleme Ağacından Arama Uzayına.... 5
3.1 Özyineleme ağacı ve çağrı yığını.... 5
3.2 Yineleme bağıntıları ve yerine koyma yöntemi.... 7
3.3 Arama uzayı, kaba kuvvet ve durum uzayı ağacı.... 8
3.4 Üretme şablonları.... 9
3.5 Geri izleme ve budama.... 10
3.6 Çıktı boyutu kadar iş ve uygulanabilirlik duvarı.... 10
4. Algoritmalar.... 11
4.1 Alt küme üretimi.... 12
4.2 Permütasyon üretimi.... 14
4.3 Alt küme toplamı (geri izleme).... 17
4.4 N-vezir (geri izleme).... 20
5. Çözümlü Örnekler.... 24
5.1 Bit maskesiyle alt kümeler: özyinelemesiz üretim (Örnek 5).... 25
5.2 k-kombinasyonlar: başlangıç indisi (Örnek 7).... 26
5.3 Labirentte yol bulma: ızgarada geri izleme (Örnek 11).... 27
5.4 Sudoku çözücüsü: kısıt yayılımı olmadan geri izleme (Örnek 13).... 28
6. Haftanın Deneyi: N-Vezirde Budamanın Etkisi.... 29
7. Hata Avcısı.... 31
8. Dil Penceresi: C · C++ · Python.... 33
9. Mühendis Gözüyle.... 36
10. Mini-OBS 2 v2.2: AKTS Aralığına Uyan Ders Kümeleri.... 37
11. Alıştırmalar ve Atölye.... 39
11.1 İzleme ve çıktı tahmini (A).... 39
11.2 Karmaşıklık (B).... 39
11.3 Değişmez yazma (C).... 40
11.4 Hata bulma (D).... 40
11.5 Parsons (E).... 41
11.6 Kod yazma (F).... 41
11.7 Diller arası okuma (G).... 42
11.8 Atölye 1 · Arama ağacı çizme (eşli, ≈ 15 dk).... 43

11.9 Atölye 2 · N-vezir yarışı: hangi budama kaç düğüm kazandırır? (gruplar, ≈ 20 dk).....43
11.10 Atölye 3 · Bulmaca çözücü (eşli, ≈ 25 dk).....44
12. Kendini Değerlendirme.....44
13. İleri Bakış ve Merak Kutusu.....45
14. Özet, Kavram Haritası ve Terim Sözlüğü.....45
15. Kaynaklar.....47
Ek A. Dil Karşılıkları.....48
A.1 Örnek 1 · Özyinelemeli toplam ve Fibonacci'de çağrı sayaçları; fib(5) çağrı ağacı.....48
A.2 Örnek 2 · Hanoi Kuleleri: T(n) = 2T(n−1) + 1 bağıntısının hamle sayacıyla doğrulanması.....49
A.3 Örnek 3 · n bitlik tüm ikili diziler: dahil et/etme ağacı.....50
A.4 Örnek 4 · Bir kümenin tüm alt kümeleri (özyinelemeli üretme şablonu).....51
A.5 Örnek 5 · Aynı problem bit maskesiyle (0 … 2^n − 1); Örnek 4 ile aynı küme ailesi ve sıra.....51
A.6 Örnek 6 · Tüm permütasyonlar — kullanildi dizisiyle, sözlük sırasında; takasla üretim.....52
A.7 Örnek 7 · k-kombinasyonlar: başlangıç indisiyle tekrarsız üretim ve C(n, k) sayımı.....53
A.8 Örnek 8 · Alt küme toplamı: kaba kuvvet ve budamalı geri izleme (ziyaret edilen düğüm).....54
A.9 Örnek 9 · N-vezir: geri izleme ve N = 1…10 için çözüm sayıları (N = 8 için 92).....55
A.10 Örnek 10 · N-vezir: sütun/köşegen dizileriyle O(1) güvenlik denetimi ve süre karşılaştırması 56
A.11 Örnek 11 · Labirentte yol bulma: ızgarada geri izleme ve yolun işaretlenmesi.....57
A.12 Örnek 12 · Özyineleme sınırı deneyi: Python RecursionError ve C'de yığın taşması.....58
A.13 Örnek 13 · Sudoku çözücüsü (meydan okuma): kısıt yayılımı olmadan geri izleme.....60

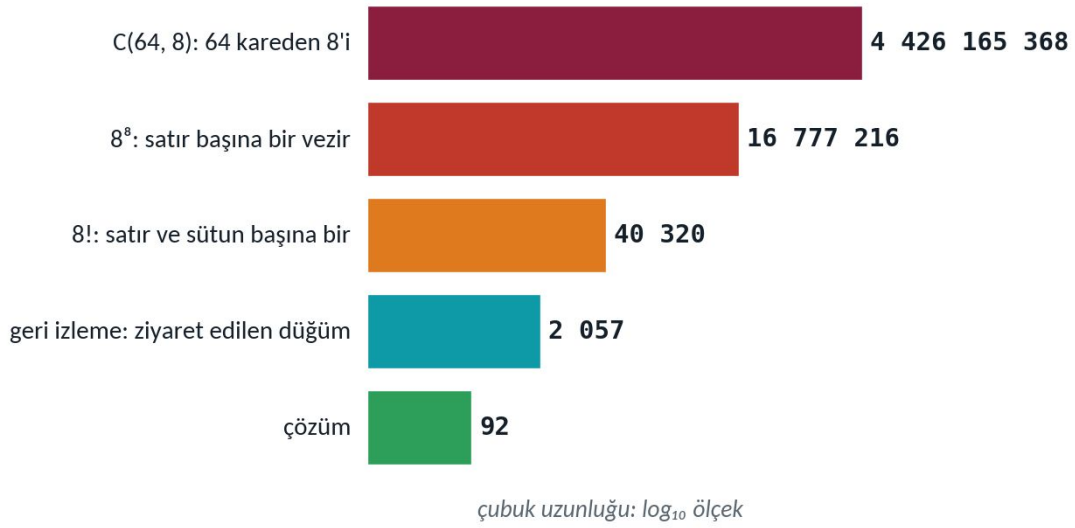
1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Satranç tahtasına birbirini tehdit etmeyen sekiz vezir kaç farklı biçimde yerleştirilebilir ve yaklaşık 4,4 milyar yerleşimi tek tek denemeden yanıtı nasıl buluruz?

Satranç tahtasının 64 karesinden 8'ini seçmenin C(64, 8) = 4 426 165 368 yolu vardır. Bunları tek tek deneyen bir program, her yerleşimde 28 vezir çiftinin birbirini tehdit edip etmediğine bakmak zorundadır. Oysa bu hafta yazacağımız geri izleme (backtracking) programı aynı soruyu yalnızca 2 057 düğüm ziyaret ederek yanıtlar ve 92 çözümün tamamını bulur. Aradaki fark daha hızlı bir bilgisayardan değil, daha akıllı bir aramadan gelir: kısmi bir yerleşim bir kurala aykırıysa onu tamamlamaya çalışmadan bırakmak, yani budamak (pruning).

Bu haftanın aracı özyinelemedir (recursion). AP-1'de özyinelemeyi bir problemi kendi küçük örneğine indirgemek için kullandık. Şimdi aynı aracı bir **arama uzayını** (search space) sistemli biçimde dolaşmak için kullanacağız: bütün alt kümeleri, bütün permütasyonları, bütün k'lı seçimleri üreteceğiz; sonra bu üretme şablonuna "olmayacak dalı kes" kuralını ekleyerek N-vezir, alt küme toplamı, labirent ve Sudoku problemlerini çözeceğiz. Yol boyunca her şeyi sayacağız: çağrı sayısı, özyineleme derinliği, ziyaret edilen ve budanan düğümler.

Şekil 3.1
8 vezir için arama uzayının daralması (sayılar h03_deney_uzay.csv'den; çubuk uzunluğu logaritmik)



GERÇEK DÜNYADAN · 1848, 1850, 1971: sekiz vezir bulmacası

Sekiz vezir bulmacası ilk kez 1848'de satranç problemcisi Max Bezzel tarafından soruldu; 92 çözüm 1850'de Franz Nauck tarafından yayımlandı. Niklaus Wirth, "adım adım iyileştirme" ile program geliştirmeyi anlattığı klasik makalesinde (Wirth, 1971) bu bulmacayı geri izleme algoritmasının örneği olarak kullandı.

Wirth'ün makalesi bizim için ayrıca bir ölçüm kıyaslaması sunar: makaleye göre program ilk çözümü bulmadan önce 876 kısmi yerleşim üretip sınar, 92 çözümün tamamını ise kareleri 15 720 kez sınayarak bulur. Bölüm 9'da kendi programlarımızın aynı iki sayıyı ölçtüğünü göreceğiz.

1.1 Öğrenme çıktıları

- **ÖÇ 3.1** Özyinelemeli bir algoritmayı özyineleme ağacıyla çizer; çağrı sayısını ve özyineleme derinliğini (bellek) belirler. (Bölüm 3.1, Örnek 1)

- **ÖÇ 3.2** Basit yineleme bağıntılarını — T(n) = T(n−1) + c, T(n) = 2T(n−1) + c, T(n) = n·T(n−1) — yerine koyma yöntemiyle çözer. (Bölüm 3.2, Örnek 2)
- **ÖÇ 3.3** Alt küme, permütasyon ve kombinasyonları özyinelemeyle ve bit maskesiyle üretir; üretilen nesne sayısını önceden hesaplar. (Bölüm 4.1–4.2, Örnek 3–7)
- **ÖÇ 3.4** Durum uzayı ağacı üzerinde geri izleme algoritmaları (N-vezir, alt küme toplamı, labirent) tasarlar; budama koşullarıyla arama uzayını daraltır. (Bölüm 4.3–4.4, Örnek 8–11, 13)
- **ÖÇ 3.5** Kaba kuvvet çözümün uygulanabilirlik sınırını (2^n için $n \approx 25$, $n!$ için $n \approx 11$) hesaplar ve gerekçelendirir. (Bölüm 3.6, Haftanın Deneyi)

2. Geçen Haftadan ve AP-1'den Bağlantı

H2'de bir algoritmanın işlem sayısını n'nin fonksiyonu olarak yazdık ve büyüme hızlarını sınıflandırdık: $\Theta(n)$, $\Theta(n \log n)$, $\Theta(n^2)$ … ve listenin en sonunda, "pratikte kaçınılması gereken" iki sınıf olarak $\Theta(2^n)$ ve $\Theta(n!)$. Bu hafta o iki sınıfın içine giriyoruz: hangi problemler doğal olarak üstel arama gerektirir, üstel arama ne zaman hâlâ uygulanabilirdir ve budama onu ne kadar küçültebilir?

AP-1 H9'dan üç kavramı yeniden kullanacağız. **Temel durum** (base case) özyinelemeyi durduran, kendini çağırmayan durumdur. **Özyinelemeli adım** (recursive step) problemi küçültüp fonksiyonu yeniden çağırır. **Çağrı yığını** (call stack) ise her çağrının parametrelerini, yerel değişkenlerini ve dönüş adresini tutan çerçevelerin (stack frame) üst üste dizildiği bellek bölgesidir: bir çağrı başladığında yığının tepesine bir çerçeve eklenir, döndüğünde kaldırılır. Bu haftanın ilk yeni fikri, bu çerçevelerin zaman içindeki öyküsünün bir **ağaç** olarak çizilebilmesidir.

> TAHMİN ET · **Geri çağırma: üç soru (yanıtlar aşağıda)**
>
> - **S1 (H2).** $\Theta(n^2)$ bir algoritmada n iki katına çıkarsa süre yaklaşık kaç katına çıkar? $\Theta(2^n)$ bir algoritmada n yalnızca 1 artarsa?
> - **S2 (AP-1 H9).** `toplam(n) = n + toplam(n − 1)`, `toplam(0) = 0` tanımıyla `toplam(3)` çağrıldığında, en derin anda yığında kaç çerçeve açıktır?
> - **S3 (AP-1 H4 ve H9).** 3 diskli Hanoi Kuleleri en az kaç hamle ister? C'de `1 << 3` ifadesinin değeri nedir?

Yanıtlar. S1: $\Theta(n^2)$'de yaklaşık 4 kat; $\Theta(2^n)$'de n'nin 1 artması süreyi yaklaşık 2 katına çıkarır (n 10 artarsa ≈ 1024 kat). S2: 4 çerçeve: toplam(3), toplam(2), toplam(1) ve toplam(0). Örnek 1 bunu ölçer: `toplam(5) = 15 çağrı = 6 en büyük derinlik = 6`. S3: $2^3 - 1 = 7$ hamle (Örnek 2); `1 << 3` = 8. İkisi de "ikiye katlanma"dır ve bu haftanın her yerinde karşımıza çıkacak.

3. Kavramsal Anlatım: Özyineleme Ağacından Arama Uzayına

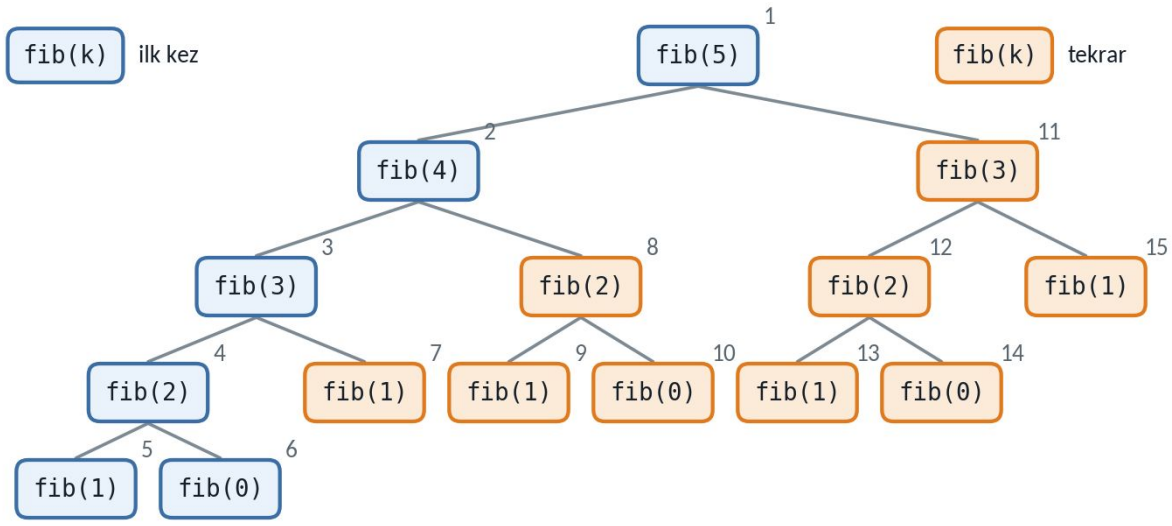
3.1 Özyineleme ağacı ve çağrı yığını

Bir özyinelemeli fonksiyonun yürütülmesini iki biçimde görebiliriz. **Özyineleme ağacı** (recursion tree) bütün çağrıları bir arada gösterir: her düğüm bir çağrıdır, bir çağrının yaptığı çağrılar onun çocuklarıdır. **Çağrı yığını** ise belirli bir **anda** hangi çağrıların henüz bitmediğini gösterir. Herhangi bir anda yığındaki çerçeveler, ağaçta kökten o an çalışan düğüme giden yolun düğümleridir. Dolayısıyla **özyineleme derinliği** (aynı anda açık en çok çerçeve) ağacın yüksekliğine, **toplam çağrı sayısı** ağaçtaki düğüm sayısına eşittir: ilki belleği, ikincisi zamanı belirler.

Örnek 1 bu iki sayıyı ölçer (kök çağrının derinliği 1): `fib(5) = 5 çağrı = 15 en büyük derinlik = 5`, `farklı alt problem = 6 tekrarlanan çağrı = 9`. fib(5) 15 kez çağrılır ama yalnızca 6 farklı alt problem vardır; kalan 9 çağrı çözülmüş bir alt problemi yeniden çözer. Ortak gösterim

sözleşmemizde tekrarlanan alt problemler aynı renkle (turuncu) çizilir (Şekil 3.2); H13'te bu israfı ara sonuçları saklayarak (memoization) kaldıracağız.

Şekil 3.2
fib(5) özyineleme ağacı: sağ üstteki numara çağrı sırası; turuncu = daha önce çözülmüş alt problemin tekrarı (Örnek 1 çıktısındaki "(tekrar)" işaretleri)



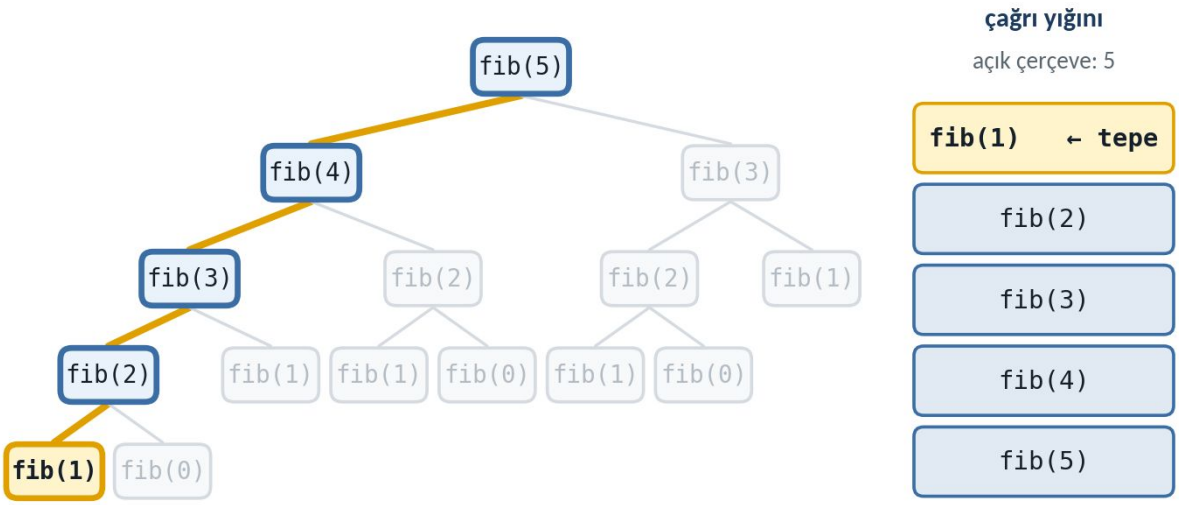
Örnek 1'deki `fib` her çağrıda sayacı artırır ve derinliği bir parametreyle (`d`, kök çağrıda 1) taşır; iki özyinelemeli çağrı ağaçtaki iki çocuktur:

c/H03_01_ozyineleme_sayac.c · satır 36–46

```c
static long long fib(int n, int d)
{
    cagri_sayisi++;
    if (d > en_buyuk_derinlik) {
        en_buyuk_derinlik = d;
    }
    if (n < 2) {
        return n;                       /* temel durum */
    }
    return fib(n - 1, d + 1) + fib(n - 2, d + 1);
}
```

Ağaç bütün yürütmeyi, yığın ise tek bir anı gösterir. Şekil 3.3, ilk kez en derine inilen anı iki görünümle birlikte verir: o anda yığında fib(5) → fib(4) → fib(3) → fib(2) → fib(1) çerçeveleri açıktır ve bunlar ağaçta kökten etkin düğüme (sarı) giden kalın yolun düğümleridir; soluk düğümler henüz çağrılmamıştır. Yığında aynı anda en çok n = 5 çerçeve bulunur: zaman ağacın düğüm sayısıyla, bellek yolun uzunluğuyla ölçülür.

Şekil 3.3
Aynı anın iki görünümü: fib(5) özyineleme ağacında kökten etkin düğüme giden yol ile çağrı yığınındaki çerçeveler (tepe üstte) aynı düğümlerdir

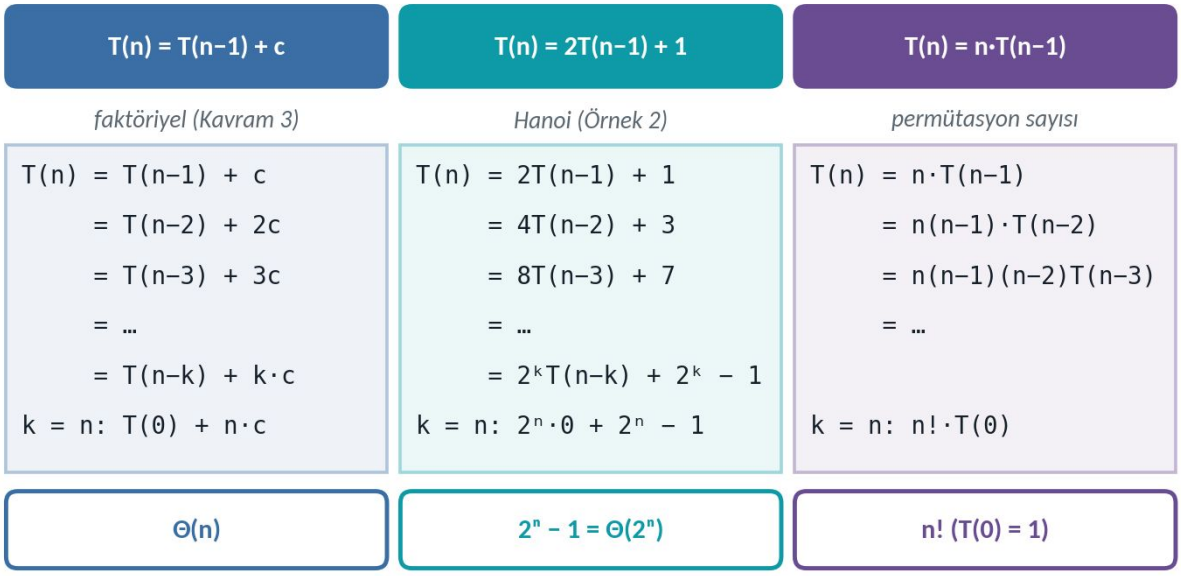


Çağrı sayısı C(n) = C(n − 1) + C(n − 2) + 1, C(0) = C(1) = 1 bağıntısına uyar ve tümevarımla C(n) = 2·F(n + 1) − 1 gösterilir; Örnek 1 eşitliği n = 0 … 25 için denetler (n = 25'te 242 785 çağrı). Derinlik ise yalnızca n'dir. Bu ayrım haftanın bütün analizlerinin temelidir: üretme ve geri izleme algoritmalarında **zaman üstel, bellek (yığın) doğrusaldır**. Her çağrı tek çağrı yaparsa ağaç bir zincirdir: `toplam(5)` 6 çağrı yapar, en derin anda 6 çerçeve açıktır.

3.2 Yineleme bağıntıları ve yerine koyma yöntemi

Özyinelemeli bir algoritmanın maliyetini bir **yineleme bağıntısıyla** (recurrence) yazarız. En basit çözüm yolu **yerine koyma** (açılım; substitution) yöntemidir: bağıntıyı kendi içine birkaç kez yerleştirip örüntüyü k adım için genel yazarız ve k'yı temel duruma ulaşacak biçimde seçeriz (Cormen vd., 2022). Bu hafta üç tip yeterlidir (Şekil 3.4): **T(n) = T(n − 1) + c** (faktöriyel, `toplam`) → Θ(n); **T(n) = 2T(n − 1) + 1** (Hanoi, ikili diziler) → $2^k T(n-k) + (2^k - 1)$, k = n'de $2^n - 1$; **T(n) = n·T(n − 1)** (permütasyon sayısı) → n!.

Şekil 3.4
Yerine koyma (açılım) yöntemiyle üç bağıntının çözümü



Hanoi Kuleleri klasik örnektir: üstteki n − 1 diski yardımcı direğe taşı, en büyüğünü hedefe koy, n − 1 diski hedefe taşı; T(n) = 2T(n − 1) + 1, T(0) = 0. Örnek 2 hamleleri listeler, açılımı yazar ve n = 1 … 20 için sayaçla ölçülen hamle sayısını bağıntı ve kapalı biçim $2^n - 1$ ile karşılaştırır (her satırda "evet"; n = 20'de 1 048 575). **Bağıntıyı yazdığımız gibi sayacı da ölçeriz:** analizin kendisi de test edilir.

C `c/H03_02_hanoi.c · satır 16–28`

```c
static void hanoi(int n, char kaynak, char hedef, char yardimci)
{
    if (n == 0) {
        return;                                 /* temel durum: disk yok */
    }
    hanoi(n - 1, kaynak, yardimci, hedef);      /* üstteki n-1 disk */
    hamle_sayisi++;                             /* en büyük disk */
    if (listele) {
        printf("%2lld. hamle: disk %d  %c -> %c\n",
               hamle_sayisi, n, kaynak, hedef);
    }
    hanoi(n - 1, yardimci, hedef, kaynak);      /* n-1 disk yeniden */
}
```

3.3 Arama uzayı, kaba kuvvet ve durum uzayı ağacı

Pek çok problem "şu koşulu sağlayan bir yapılandırma bul" biçimindedir. Bütün aday yapılandırmaların kümesine **arama uzayı** (search space) denir; **kaba kuvvet** (brute force) bu uzayın her elemanını üretip denetler. Doğruluğu kolayca gösterilir (her aday denendiyse hiçbir çözüm kaçmamıştır), ama maliyeti uzayın büyüklüğüdür. n = 20 için: alt kümeler 2^{20} = 1 048 576, yarı boy kombinasyonlar C(20, 10) = 184 756 (Örnek 7), permütasyonlar 20! ≈ $2{,}4 \cdot 10^{18}$ (Kavram 3).

Uzayı sistemli dolaşmak için onu bir ağaç olarak düşünürüz: kökte hiçbir karar verilmemiştir, her düzeyde bir karar verilir (elemanı al/alma, sıradaki konuma şunu koy), yapraklar tam yapılandırmalardır. Bu **durum uzayı ağacını** (state-space tree) özyinelemeli fonksiyon **derinlik öncelikli** (depth-first) dolaşır; ağacın tamamı hiçbir zaman bellekte değildir, yalnızca kökten etkin düğüme giden

yol (yığın) tutulur. En basit örnek n bitlik ikili dizilerdir: Örnek 3'teki `ikili_uret` sol dalda biti 0, sağ dalda 1 yapar.

c/H03_03_ikili_diziler.c · satır 20–35

```c
static void ikili_uret(int k, int n)
{
    cagri_sayisi++;
    if (k == n) {                   /* yaprak: dizi tamamlandı */
        dizi_sayisi++;
        if (yazdir_mi) {
            dizi[n] = '\0';
            printf("%s\n", dizi);
        }
        return;
    }
    dizi[k] = '0';                  /* sol dal: k. bit 0 */
    ikili_uret(k + 1, n);
    dizi[k] = '1';                  /* sağ dal: k. bit 1 */
    ikili_uret(k + 1, n);
}
```

Çıktı `000 001 010 … 111` ve `dizi = 8 çağrı = 15`: sol dal önce gezildiği için diziler sözlük sırasında çıkar; n bit için 2^n yaprak ve $2^{n+1} - 1$ çağrı vardır (n = 1 … 16 için denetlenir). `dizi[k]`'ya yazılan bit kardeş dala geçilirken **üzerine yazıldığı** için ayrıca "geri al" gerekmez; Örnek 4'te seçilenleri bir yığında tuttuğumuzda geri almayı açıkça yazacağız.

3.4 Üretme şablonları

Bu haftanın bütün üretme algoritmaları, "şu ana kadar verilen kararları" bir durumda taşıyıp bir sonraki kararın her seçeneğini dener ve üç şablondan birine uyar:

- **Dahil et / etme** (include–exclude): her eleman için iki dal; ağaç ikili, derinlik n, yaprak 2^n. Alt kümeler, ikili diziler, alt küme toplamı (Örnek 3, 4, 8).
- **Seç – dene – geri al** (choose–explore–unchoose): sıradaki konum için kullanılabilir her seçeneği seç, dene, geri al. Permütasyonlar, N-vezir, labirent, Sudoku (Örnek 6, 9–11, 13).
- **Başlangıç indisi** (start index): bir sonraki seçim yalnızca son seçilenden sonraki elemanlardan yapılır; her küme tam bir kez üretilir. k-kombinasyonlar (Örnek 7), Mini-OBS 2 v2.2.

Hepsini birleştiren geri izleme iskeleti aşağıdadır; farklılaşan yalnızca "durum", "aday" ve "uyumlu" kavramlarıdır (Skiena, 2020; Levitin, 2012).

SÖZDE KOD · Geri izleme şablonu

```
algoritma GERİ-İZLE(durum)
    eğer durum bir çözüm ise
        çözümü kaydet ya da say; döndür
    her aday x için                          ▷ sabit bir sırayla
        eğer x durumla uyumlu ise            ▷ değilse BUDA: alt ağaç açılmaz
            durum ← durum + x                ▷ seç
            GERİ-İZLE(durum)                 ▷ dene
            durum ← durum − x                ▷ geri al
```

3.5 Geri izleme ve budama

Geri izleme (backtracking), durum uzayı ağacını derinlik öncelikli dolaşırken bir kısmi çözümün hiçbir tam çözüme genişletilemeyeceği anlaşıldığı anda o dalı bırakıp bir önceki karar noktasına dönen arama yöntemidir. Wirth (1971) sekiz vezir programını bu adla anlatır; Dijkstra da yapısal programlama notlarında aynı bulmacayı kullanır (Dahl vd., 1972). Bırakılan dala **budanmış** (pruned) denir; torunlarının hiçbiri üretilmez.

Budamanın bir bedeli ve bir sorumluluğu vardır. **Bedel:** her düğümde kısıt denetimi yapılır; denetim ucuz olmalıdır (N-vezirde O(1), Bölüm 4.4). **Sorumluluk:** budama **güvenli** olmalı, hiçbir çözümü atmamalıdır; her kural küçük bir kanıta dayanır: "bu düğümün hiçbir torunu çözüm olamaz, çünkü …". Alt küme toplamında "toplam > hedef ise dur" kuralı **bütün elemanlar pozitifken** güvenlidir; dizide negatif sayı varsa kanıt çöker ve kural çözüm kaçırır (Hata Avcısı 4).

> KAPUTUN ALTINDA · **Geri almanın bellek maliyeti: kopyalamak mı, geri almak mı?**
>
> Durumu her çağrıya **kopyalayarak** geçirseydik geri alma gerekmezdi, ama her düğümde O(n) kopya ve derinlik boyunca $O(n^2)$ bellek öderdik. Bu haftanın programları durumu **yerinde değiştirip geri alır** (`yol[uzunluk++] = a[i]` … `uzunluk--`; `kullanildi[i] = 1` … `kullanildi[i] = 0`): düğüm başına O(1) iş, O(n) ek bellek. Kopya yalnızca bir çözüm **saklanacaksa**, yaprakta gerekir (Python'da unutulursa: Hata Avcısı 7).

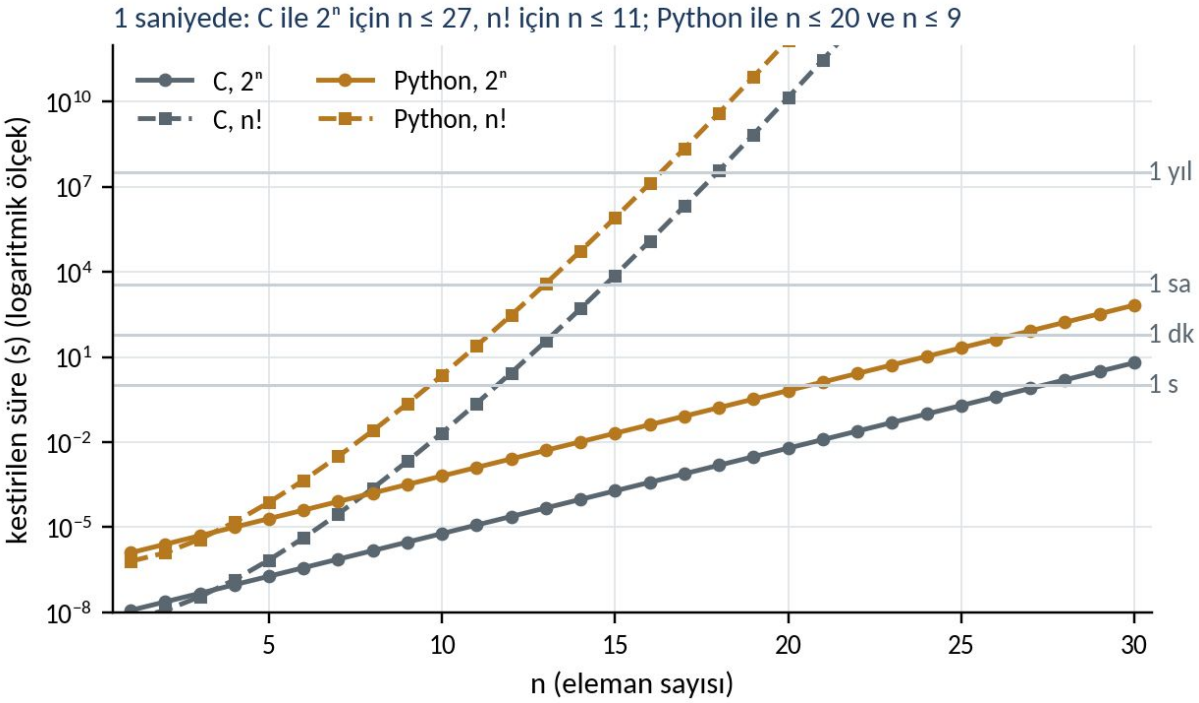
3.6 Çıktı boyutu kadar iş ve uygulanabilirlik duvarı

Bütün alt kümeleri **listeleyen** bir algoritma en az listenin kendisini yazacak kadar çalışmak zorundadır: 2^n alt kümenin eleman sayıları toplamı $n \cdot 2^{n-1}$'dir (Örnek 4, n = 16: 65 536 küme, 524 288 eleman). Dolayısıyla listeleme $\Omega(n \cdot 2^n)$ zaman ister ve $\Theta(n \cdot 2^n)$ algoritmamızdan **daha hızlısı olamaz**; permütasyonlar için sınır $\Omega(n \cdot n!)$'dir. Çıktı üstelse hiçbir budama işi polinoma indiremez; budama ancak çıktı arama uzayından çok küçükken işe yarar.

Uzayın büyüklüğü, kod yazılmadan önce kaba kuvvetin uygulanabilir olup olmadığını söyler. Bu makinede budamasız N-vezirin ölçülen hızıyla (C -O2 ≈ $1{,}7 \cdot 10^8$, Python ≈ $1{,}6 \cdot 10^6$ düğüm/s) 1 saniyede C ile 2^n nesne için n ≤ 27, n! için n ≤ 11; Python ile n ≤ 20 ve n ≤ 9. Bir dakika sınırları yalnızca birkaç adım öteler (C: 33 ve 13). ÖÇ 3.5'teki "2^n için n ≈ 25, n! için n ≈ 11" kuralının ölçülmüş karşılığı budur (Şekil 3.5).

Derinlik ve çağrı sayısı ayrı büyür. Permütasyon üretiminde aynı anda en çok n + 1 çerçeve açıktır, ama çağrı sayısı $n!/0! + n!/1! + \ldots + n!/n! = \lfloor e \cdot n! \rfloor$'dir (n ≥ 1); Örnek 6 n = 8 için 109 601 çağrı ölçer.

Şekil 3.5
Uygulanabilirlik duvarı: 2^n ve $n!$ nesneyi tek tek işlemenin kestirilen süresi (bu makinenin düğüm hızıyla, nesne başına bir düğüm; y logaritmik)



KAPUTUN ALTINDA · **Yığın çerçevesi, -O2 ve Python'un çerçeveleri (bu makinede ölçüldü)**

Çerçeve. GCC 13 -O0 ile derlenen `toplam(n)` (Örnek 12) çağrı başına 32 bayt kullanır: 8 bayt dönüş adresi, 8 bayt kaydedilen rbp, 16 bayt yerel alan (`subq $16, %rsp`). 8 MB yığında (`ulimit -s` = 8192) bu ≈ 262 144 çerçevedir; ölçülen ilk çökme derinliği 261 213'tür.

-O2. Kavram 6'da `toplam` ve kuyruk özyinelemeli (tail recursion) `toplam_kuyruk`, -O0'da n = 1 000 000 için SIGSEGV ile çöker (kabukta 139); -O2'de n = 10 000 000 için doğru sonuç verir ve assembly'de `call toplam` kalmaz. Dönüşümü GCC'nin `-foptimize-sibling-calls` eniyilemesi yapar: bu makinede (GCC 13) -O2, -O3 ve -Os'de açık, -O0 ve -O1'de kapalıdır (`gcc -Q --help=optimizers`); `-O2 -fno-optimize-sibling-calls` ile `call toplam` geri gelir. `toplam` kuyruk özyinelemeli değildir (toplama çağrıdan sonra yapılır), ama toplama birleşmeli olduğu için GCC ara toplamı bir birikim değişkenine taşıyarak onu da döngüye çevirir. C standardı bu eniyilemeyi güvence etmez, başka derleyicide ya da düzeyde sonuç değişebilir; Örnek 12 bu yüzden -O0 ile derlenir.

Python. CPython her Python çağrısı için bir çerçeve tutar; `f_back` bağı çağıranın çerçevesini gösterir ve Kavram 7 bu zinciri yürür: `toplam(n=0) <- toplam(n=1) <- toplam(n=2) <- toplam(n=3) <- main <- <module>`. Python 3.11'den beri bu çerçeveler hafif iç yapılardır, çerçeve nesnesi yalnızca istendiğinde (ör. `sys._getframe()`) oluşturulur ve Python'dan Python'a çağrılar C yığınını kullanmaz (Python Software Foundation, t.y.-b). `sys.getrecursionlimit()` = 1000 aynı anda açık Python çerçevelerinin sayısını sınırlar (Bölüm 8).

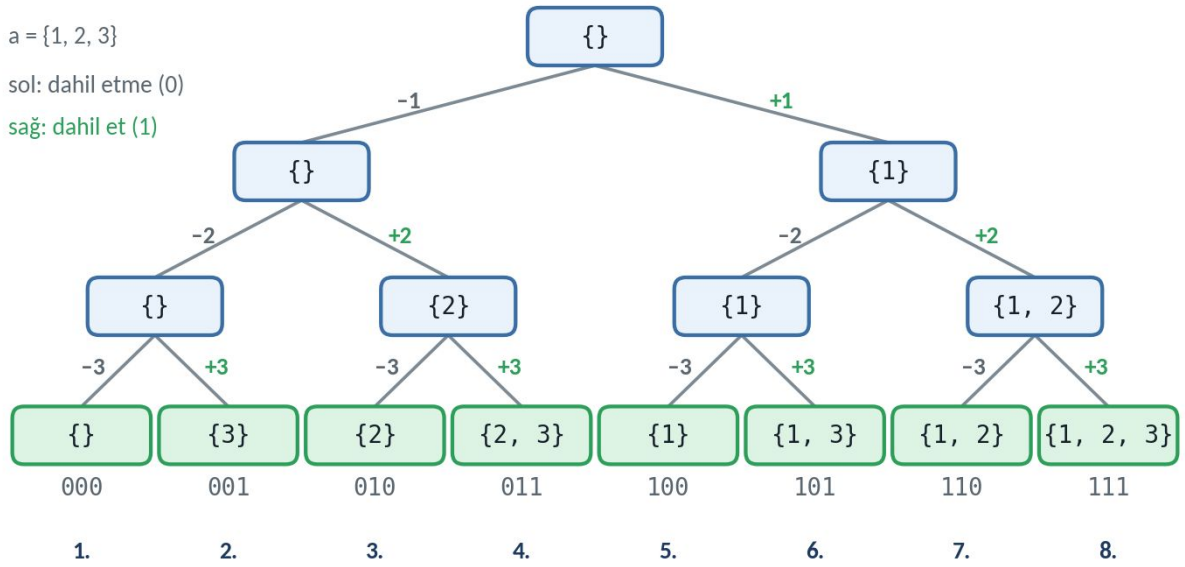
4. Algoritmalar

Dört amiral algoritmayı aynı on bir adımlı standart anlatımla işliyoruz: problem, fikir, elle izleme, sözde kod, doğruluk, analiz, C, üç dil, test, deney ve varyantlar. İlk ikisi üretme, son ikisi geri izleme algoritmasıdır; dördü de Bölüm 3.4'teki şablonun özel durumlarıdır.

4.1 Alt küme üretimi

1 · Problem. Girdi: n elemanlı a[0..n − 1]. Çıktı: bütün alt kümeler, her biri tam bir kez; a = {1, 2, 3} için 2^3 = 8 küme. **2 · Fikir.** Her eleman için bir evet/hayır kararı verilir; kararlar bir ikili ağaç oluşturur: sol dal "a[i] alınmaz", sağ dal "a[i] alınır" (Şekil 3.6). Seçilenler bir yığında (`yol`) biriktirilir: sağ dala girerken eleman eklenir (seç), dal bitince çıkarılır (geri al).

Şekil 3.6
a = {1, 2, 3} için dahil et/etme ağacı: düğümde o ana kadar seçilenler; yaprak altında karar dizisi (a[0] soldaki bit) ve üretim sırası



3 · Elle izleme. Ağaç ön sırayla dolaşılır: 15 çağrı; yapraklarda sırasıyla {}, {3}, {2}, {2, 3}, {1}, {1, 3}, {1, 2}, {1, 2, 3}. Her an `yol`, kökten o düğüme giden yoldaki "+" kararlarıdır.

> **TAHMİN ET · Sizce sırada ne var?**
>
> {2, 3} yazıldıktan sonra hangi çerçeveler biter ve 5. küme hangisidir? (Yanıt: i = 3, 2 ve 1 çerçeveleri biter, 3 ve 2 yoldan çıkar; kök sağ dalına geçer, 1 seçilir ve {1} yazılır.)

SÖZDE KOD · 4 · Sözde kod: dahil et / etme

```
algoritma ALT-KÜMELER(i)                ▷ a[0..i−1] için karar verildi
    eğer i = n ise
        yol'u yaz; döndür               ▷ yaprak: bir alt küme
    ALT-KÜMELER(i + 1)                  ▷ a[i] alınmaz
    yol'un sonuna a[i] ekle             ▷ seç
    ALT-KÜMELER(i + 1)                  ▷ dene: a[i] alınır
    yol'un son elemanını çıkar          ▷ geri al
```

5 · Doğruluk. Değişmez (invariant; Tablo 3.1): alt_kumeler(i) çağrısında yol, a[0..i−1] için verilmiş kararlarda seçilen elemanları tam olarak içerir; çağrı döndüğünde yol, çağrıdan önceki hâline döner (geri al). Sağ dal eklediğini dönüşte çıkarır; i her çağrıda arttığından i = n'de durulur. Her yaprak bir (b_0, …, b_{n-1}) karar dizisiyle birebir eşlenir: tekrar da eksik de yoktur. **6 · Analiz.** 2^n yaprak, $2^{n+1} - 1$ çağrı, yaprakta O(n) yazma: her girdide $\Theta(n \cdot 2^n)$, ek bellek O(n); Bölüm 3.6'daki alt sınır gereği daha hızlısı olamaz.

Tablo 3.1
Alt küme üretimi kimlik kartı

ALGORİTMA KİMLİK KARTI **Alt küme üretimi** *(subset generation (power set))*	
Strateji	Kaba kuvvet (özyinelemeli üretme: dahil et / etme ağacı; özyinelemesiz eşi: bit maskesi 0 … $2^n - 1$)
Girdi önkoşulu	Elemanlar ayrık kabul edilir; n küçük olmalı (2^n küme: kaba kuvvet için n ≈ 25'e kadar)
Zaman (en iyi · ortalama · en kötü)	$\Theta(n \cdot 2^n)$ · $\Theta(n \cdot 2^n)$ · $\Theta(n \cdot 2^n)$
Ek bellek	O(n) yığın
Özellikler	Çıktı boyutu kadar iş: 2^n küme, toplam $n \cdot 2^{n-1}$ eleman; daha hızlısı olamaz. Ağaçta 2^n yaprak ve $2^{n+1} - 1$ çağrı; özyineleme derinliği n + 1. "Dahil etme" dalı önce gezilirse kümeler ikili sayma sırasında çıkar (a[0] en soldaki bit)
Değişmez / doğruluk	alt_kumeler(i) çağrısında yol, a[0..i−1] için verilmiş kararlarda seçilen elemanları tam olarak içerir; çağrı döndüğünde yol, çağrıdan önceki hâline döner (geri al)
Kütüphane karşılığı	C: yok (elle; bit maskesi döngüsü) · C++: yok (`std::bitset` yalnızca gösterim) · Python: `itertools.product([0, 1], repeat=n)`; her r için `itertools.combinations(a, r)` (yalnızca kâhin)
Ne zaman kullanılır	Tüm seçeneklerin denenmesi gereken küçük problemlerde, daha hızlı algoritmaların kâhini olarak, alt küme toplamı / sırt çantası gibi problemlerin ilk doğru çözümü olarak
Ne zaman kullanılmaz	n büyükken (n = 40'ta $2^{40} \approx 1{,}1 \cdot 10^{12}$ küme); yalnızca koşulu sağlayan kümeler aranıyorsa budamalı geri izleme, en iyi küme aranıyorsa dinamik programlama (H13)

7 · C gerçekleştirimi. (Örnek 4; seçilenler bir dizi ve `uzunluk` sayacıyla tutulur):

C `c/H03_04_alt_kumeler.c · satır 38–49`

```c
static void alt_kumeler(int i)
{
    cagri_sayisi++;
    if (i == n) {          /* yaprak */
        kume_isle();
        return;
    }
    alt_kumeler(i + 1);    /* a[i] yok */
    yol[uzunluk++] = a[i]; /* seç */
    alt_kumeler(i + 1);    /* dene */
    uzunluk--;             /* geri al */
}
```

8 · Üç dil karşılaştırması. C++ fonksiyonu C'ninkiyle yalnızca yığın işlemlerinde ayrışır: `yol` bir `std::vector<int>`'tir (`push_back`/`pop_back`; aşağıda yalnızca bu dört satır). Python'da `yol` bir listedir (`append`/`pop`) ve modül düzeyi sayaç `global` bildirimi ister. Hardal satırlar C'den anlam olarak farklılaşan satırlardır:

C++ **H03_04_alt_kumeler.cpp · satır 44–47**

```cpp
    alt_kumeler(i + 1);    // a[i] yok
    yol.push_back(a[i]);   // seç
    alt_kumeler(i + 1);    // dene
    yol.pop_back();        // geri al
```

Python **H03_04_alt_kumeler.py · satır 31–40**

```python
def alt_kumeler(i):
    global cagri_sayisi
    cagri_sayisi += 1
    if i == n:             # yaprak
        kume_isle()
        return
    alt_kumeler(i + 1)     # a[i] yok
    yol.append(a[i])       # seç
    alt_kumeler(i + 1)     # dene
    yol.pop()              # geri al
```

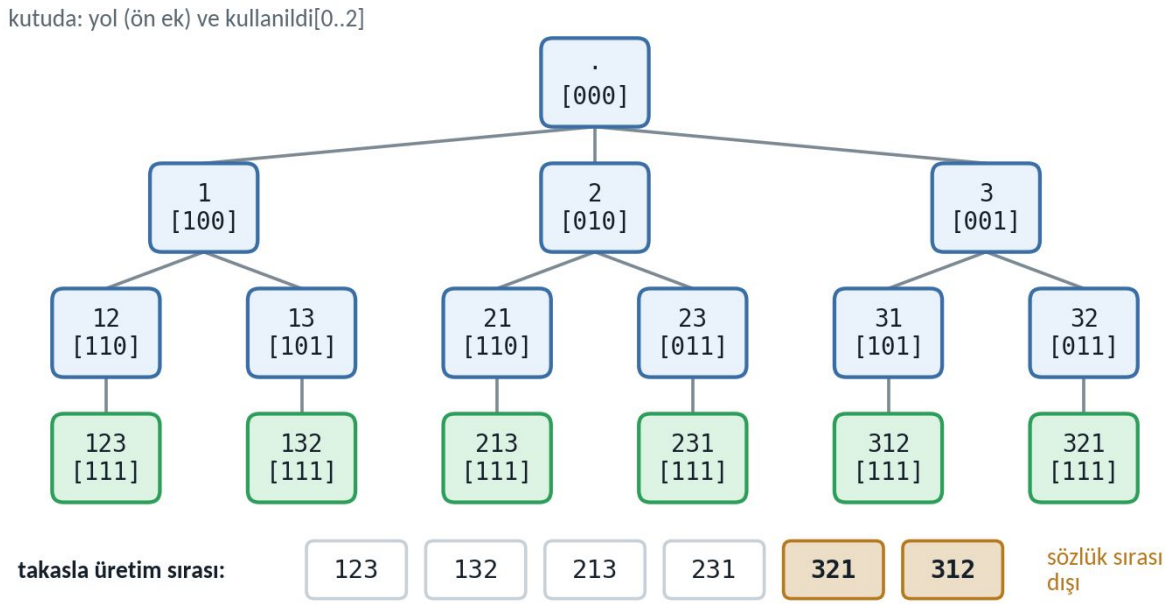
> DİL PENCERESİ · **Farklılaşan satırlar**
>
> C'de seçilenler sabit dizi ve `uzunluk` sayacıyla tutulur, geri alma `uzunluk--` ile yapılır; C++'ta `push_back`/`pop_back`, Python'da `append`/`pop` aynı yığın davranışını verir. Geri alma adımı üç dilde de özyinelemeli çağrının hemen ardındadır.

9 · Test. Program n = 1 … 16 için küme sayısını (2^n), çağrı sayısını ($2^{n+1} - 1$) ve eleman toplamını ($n \cdot 2^{n-1}$) denetler; sınır durumu n = 0'da tek küme {} vardır. Kâhin (test oracle): Örnek 5 aynı aileyi `maske = 0 … (1u << n) - 1` döngüsüyle üretir ve her yaprağı bir maskeyle karşılaştırır (a[0] soldaki bit); Python'da `itertools.product` (Kavram 1). **10 · Deney.** n = 16'da `16 65536 131071 524288 524288 evet`; n bir artınca sayılar tam iki katına çıkar. **11 · Varyantlar.** Dallar yer değiştirirse sıra değişir (quiz); bit maskesiyle özyinelemesiz üretim (5.1); yalnızca k elemanlılar için başlangıç indisi (5.2); toplam koşuluyla budama (4.3); Python'da üreteç (8).

4.2 Permütasyon üretimi

1 · Problem. Artan sıralı n eleman (1, 2, …, n) → bütün n! sıralama, **sözlük sırasında** (lexicographic order); n = 3: 123 132 213 231 312 321. **2 · Fikir.** Permütasyon konum konum kurulur: k. konuma kullanılmamış her eleman sırayla **seçilir** (`kullanildi[i] = 1`, `yol[k] = a[i]`), k + 1. konum **denenir**, sonra seçim **geri alınır**. Adaylar artan sırada denendiği için çıktı sözlük sırasındadır.

Şekil 3.7
n = 3 permütasyon ağacı: kutuda yol (ön ek) ve kullanildi dizisi; altta takasla üretimin sırası (321 ve 312 sözlük sırası dışında)



> **TAHMİN ET · Sizce sırada ne var?**
>
> 132 yazıldıktan sonra kullanildi hangi adımlardan geçer, üçüncü permütasyon hangisidir? (Yanıt: 2, 3 ve 1'in işaretleri sırayla kalkar ([0, 0, 0]); kökte i = 1 seçilir: [0, 1, 0] → [1, 1, 0] → [1, 1, 1] ve 213 yazılır.)

SÖZDE KOD · 4 · Sözde kod: kullanildi dizisiyle permütasyonlar

```
1  algoritma PERMÜTASYON(k)                ▷ yol[0..k−1] dolu
2      eğer k = n ise
3          yol'u yaz; döndür
4      i ← 0'dan n−1'e kadar her i için
5          eğer kullanildi[i] = 0 ise
6              kullanildi[i] ← 1; yol[k] ← a[i]   ▷ seç
7              PERMÜTASYON(k + 1)                 ▷ dene
8              kullanildi[i] ← 0                  ▷ geri al
```

5 · Doğruluk. Değişmez (Tablo 3.2): permutasyon(k) çağrısında yol[0..k−1] birbirinden farklı elemanlardır ve kullanildi[i] = 1 ⇔ a[i] yol[0..k−1] içindedir; seç ile geri al simetrik olduğu için korunur. Kardeşler k. konumda farklı elemanlar taşır (tekrar yok), her uygun eleman denenir (eksik yok) ve artan sırada gezilir (sözlük sırası). **6 · Analiz.** k derinliğinde n!/(n − k)! düğüm vardır; toplam ⌊e·n!⌋ çağrı (n = 8: 109 601). İç düğüm n aday tarar, yaprak n eleman yazar: Θ(n·n!). Ek bellek O(n); derinlik n + 1.

Tablo 3.2
Permütasyon üretimi kimlik kartı

ALGORİTMA KİMLİK KARTI **Permütasyon üretimi** *(permutation generation)*	
Strateji	Kaba kuvvet (geri izleme şablonuyla üretme: seç → dene → geri al; kullanildi dizisi)
Girdi önkoşulu	Sözlük sırası için girdi artan sıralı olmalı; tekrar eden eleman varsa ek kural gerekir (Hata Avcısı 3)

Zaman (en iyi · ortalama · en kötü)	Θ(n·n!) · Θ(n·n!) · Θ(n·n!)
Ek bellek	O(n)
Özellikler	Özyineleme derinliği n + 1; toplam çağrı n!/0! + n!/1! + … + n!/n! = ⌊e·n!⌋ ≈ 2,718·n! (n = 8: 40 320 permütasyon, 109 601 çağrı). Takasla üretim aynı n! permütasyonu sözlük sırası dışında verir
Değişmez / doğruluk	permutasyon(k) çağrısında yol[0..k−1] birbirinden farklı elemanlardır ve kullanildi[i] = 1 ⇔ a[i] yol[0..k−1] içindedir
Kütüphane karşılığı	C: yok (elle) · C++: `std::next_permutation` (sözlük sırasında sonraki) · Python: `itertools.permutations(a)` (yalnızca kâhin)
Ne zaman kullanılır	n ≤ ~10–11 için tüm sıralamaların denenmesi (sıralama/çizelgeleme kaba kuvveti, kâhin testleri)
Ne zaman kullanılmaz	n ≥ 12 civarında (12! ≈ 4,8·10⁸ permütasyon); yalnızca sayı gerekiyorsa n! formülü yeter

7 · C gerçekleştirimi. (Örnek 6; aynı dosyadaki `permutasyon_takas` ağacı takasla kurar, sözlük sırası bozulur: Ek A.6):

C `c/H03_06_permutasyonlar.c · satır 49–64`

```c
static void permutasyon(int k)
{
    cagri_sayisi++;
    if (k == n) {          /* yaprak */
        perm_isle(yol);
        return;
    }
    for (int i = 0; i < n; i++) {
        if (!kullanildi[i]) {
            kullanildi[i] = 1; /* seç */
            yol[k] = a[i];
            permutasyon(k + 1); /* dene */
            kullanildi[i] = 0; /* geri al */
        }
    }
}
```

8 · Üç dil karşılaştırması. `permutasyon` C++'ta C'ninkiyle aynıdır; takaslı sürümde C'deki yardımcı `takas` yerine `std::swap` kullanılır (aşağıdaki C++ kesiti). Python'da adaylar `range(n)` ile taranır, bayraklar `True`/`False`'tur. İnce bir fark daha: `perm_isle`'de C++ `onceki = p;` vektörü **kopyalar** (C++ satır 38); Python'da aynı yazım yalnızca ikinci bir ad verir, bu yüzden `onceki = p[:]` yazılır (Python satır 30; Tshukudu & Cutts, 2020):

C++ `H03_06_permutasyonlar.cpp · satır 73–77`

```cpp
    for (int i = k; i < n; i++) {
        std::swap(a[k], a[i]);     // seç
        permutasyon_takas(k + 1);  // dene
        std::swap(a[k], a[i]);     // geri al
    }
```

Python **H03_06_permutasyonlar.py · satır 38–49**

```python
def permutasyon(k):
    global cagri_sayisi
    cagri_sayisi += 1
    if k == n:              # yaprak
        perm_isle(yol)
        return
    for i in range(n):
        if not kullanildi[i]:
            kullanildi[i] = True   # seç
            yol[k] = a[i]
            permutasyon(k + 1)     # dene
            kullanildi[i] = False  # geri al
```

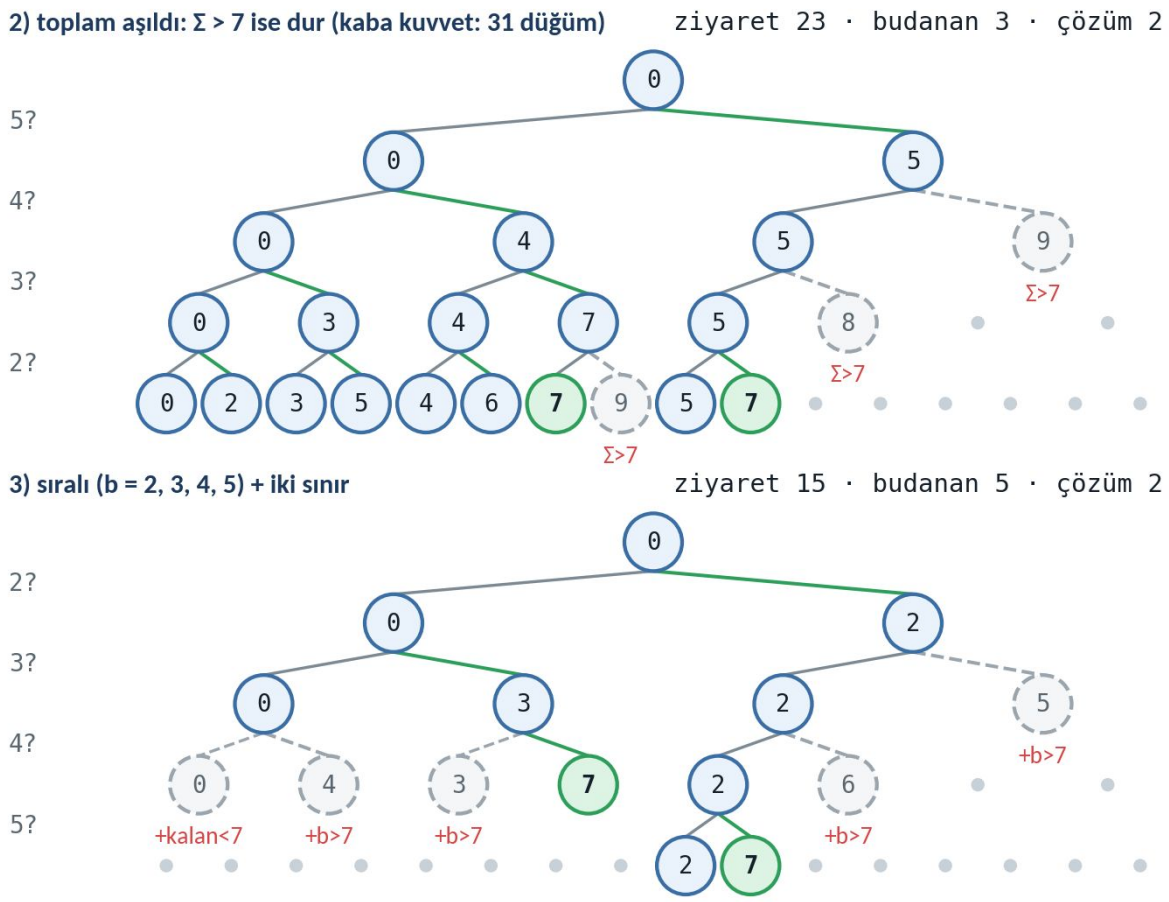
> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C++'ta `onceki = p` vektörü kopyalar; Python'da aynı yazım yalnızca aynı listeye ikinci bir ad verir, bu yüzden `p[:]` ile kopya alınır (benzer sözdizimi, farklı anlam). Takas C'de yardımcı fonksiyonla, C++'ta `std::swap`, Python'da çoklu atamayla yapılır.

9 · Test. Program n = 3 için `123 132 213 231 312 321` yazar ve her n = 1 … 8 için sayının n!, sıranın sözlük sırası olduğunu denetler. Kâhinler: C++ `std::next_permutation` (Kavram 2: `{1, 2, 3}: 123 132 213 231 312 321`; tekrar eden elemanda her diziliş bir kez: `{1, 1, 2}: 112 121 211`) ve Python `itertools.permutations` (Kavram 1, n = 1 … 7). **10 · Deney.** Çağrı/n! oranı e'ye yakınsar: n = 8'de 109 601/40 320 = 2,7183. **11 · Varyantlar.** Takasla üretim; tekrar eden elemanlar (Hata Avcısı 3); k-permütasyonlar; N-vezir, permütasyon üretiminin budamalı hâlidir (4.4).

4.3 Alt küme toplamı (geri izleme)

1 · Problem. Pozitif tamsayılar a[0..n − 1] ve bir hedef → toplamı hedefe eşit bütün alt kümeler; a = {5, 4, 3, 2}, hedef = 7 için {4, 3} ve {5, 2}. **2 · Fikir.** Alt küme ağacında bir düğümde toplam hedefi aştıysa, sayılar pozitif olduğundan hiçbir torun çözüm olamaz: dal bırakılır. Toplam **parametre olarak** taşınır (`toplam + a[i]`), böylece düğüm başına iş O(1)'dir ve geri alma kendiliğinden olur.

Şekil 3.8
a = {5, 4, 3, 2}, hedef = 7: iki budamalı yöntemin ağaçları. Düğümde o anki toplam; yeşil kenar "alınır"; kesikli gri = budanan düğüm (çağrılır ama genişletilmez), nokta = hiç çağrılmayan düğüm



3 · Elle izleme. Kaba kuvvet 31 düğümün hepsini ziyaret eder. "Toplam aşıldı" üç düğümde durur ({4, 3, 2} Σ=9; {5, 3} Σ=8; {5, 4} Σ=9) ve 23 düğüm ziyaret eder. Sıralı + iki sınır yöntemi b = (2, 3, 4, 5) üzerinde çalışır, beş düğümü budar ve 15 düğüm ziyaret eder; örneğin {} Σ=0 düğümünde "toplam + kalan[3] = 5 < 7".

> **TAHMİN ET · Sizce budanır mı?**
>
> Hedef 9 olsaydı "toplam aşıldı" {5, 4} düğümünü (Σ = 9) budar mıydı? (Yanıt: hayır; 9 > 9 yanlıştır, {5, 4} bir çözümdür. >= yazmak toplamı tam hedef olan çözümleri kaçırır: Alıştırma 3.8.)

SÖZDE KOD · 4 · Sözde kod: "toplam aşıldı" budamalı geri izleme

```
1  algoritma AKT(i, toplam)                ▷ toplam = yol'daki elemanların toplamı
2      eğer toplam > hedef ise              ▷ budama: her a[i] > 0
3          budanan ← budanan + 1; döndür
4      eğer i = n ise
5          eğer toplam = hedef ise çözümü kaydet
6          döndür
7      AKT(i + 1, toplam)                   ▷ a[i] alınmaz
8      AKT(i + 1, toplam + a[i])            ▷ a[i] alınır
```

5 · Doğruluk. Değişmez (Tablo 3.3): Her çağrıda toplam, yol'daki (a[0..i−1] arasından seçilmiş) elemanların toplamına eşittir. **Budamanın güvenliği:** toplam > hedef ise torunlar bu değere negatif olmayan sayılar ekler, hepsi > hedef kalır; kanıt "her a[i] ≥ 0" varsayımına dayanır (Hata Avcısı 4). Sıralı yöntemde toplam + b[i] > hedef ise en küçük kalan eleman bile sığmaz, toplam + kalan[i] < hedef ise kalanların hepsi yetmez. **6 · Analiz.** En kötü durumda O(2ⁿ); kazanç girdiye bağlıdır. n = 18 rastgele sayıda (tohum 42) tam ağaç 524 287 düğümken hedef 50 için 3 967 (sıralı: 937), hedef 187 için 341 657 düğüm ziyaret edilir: çözüm çoksa (2 090) toplam ancak derinde aşılır.

Tablo 3.3
Alt küme toplamı (geri izleme) kimlik kartı

ALGORİTMA KİMLİK KARTI	**Alt küme toplamı (geri izleme)** *(subset sum (backtracking))*
Strateji	Geri izleme (budamalı; dahil et / etme ağacı)
Girdi önkoşulu	"Toplam aşıldı" budaması için tüm sayılar negatif olmamalı (Hata Avcısı 4); sıralı girdi ek budama sağlar
Zaman (en iyi · ortalama · en kötü)	— · — · O(2ⁿ)
Ek bellek	O(n)
Özellikler	Toplam parametreyle taşındığı için düğüm başına O(1) iş. Bu makinede n = 18, hedef 50: kaba kuvvet 524 287, "toplam aşıldı" 3 967, sıralı + iki sınır 937 düğüm; hedef 187 iken 524 287 / 341 657 / 140 885 — kazanç hedefe bağlıdır
Değişmez / doğruluk	Her çağrıda toplam, yol'daki (a[0..i−1] arasından seçilmiş) elemanların toplamına eşittir
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (kâhin: bit maskesiyle 0 … 2ⁿ − 1 taraması)
Ne zaman kullanılır	n ≤ ~25–30 iken tüm çözümler isteniyorsa; hedef toplama göre küçükse budama çok etkilidir
Ne zaman kullanılmaz	Sayılar ve hedef küçük tamsayılarsa O(n·hedef) dinamik programlama (H13); negatif sayı varsa "toplam aşıldı" budaması

7 · C gerçekleştirimi. (Örnek 8; kaba kuvvetten yalnızca 61–64. satırlarla ayrışır):

`c/H03_08_alt_kume_toplami.c · satır 58–75`

```c
static void budamali(int i, long long toplam)
{
    ziyaret++;
    if (toplam > hedef) {
        budanan++;                          /* torunların hepsi de aşar */
        return;
    }
    if (i == n) {
        if (toplam == hedef) {
            cozum_isle();
        }
        return;
    }
    budamali(i + 1, toplam);
    yol[uzunluk++] = a[i];
    budamali(i + 1, toplam + a[i]);
    uzunluk--;
}
```

8 · Üç dil karşılaştırması. Budamalı fonksiyon C++'ta C'ninkiyle satır satır aynıdır (yalnızca `yol` bir vektördür; C++ kaba kuvvet fonksiyonu Ek A.8'de). C++'taki asıl fark girdinin üretilmesindedir (aşağıdaki iki satır); Python sürümü tam olarak verilmiştir:

C++ H03_08_alt_kume_toplami.cpp · satır 155–156

```cpp
    rastgele_tohum(42);
    a = rastgele_dizi(n, 1, 40);
```

Python H03_08_alt_kume_toplami.py · satır 44–58

```python
def budamali(i, toplam):
    """2) Geri izleme: toplam hedefi aştıysa dur (her a[i] > 0 olduğu için)"""
    global ziyaret, budanan
    ziyaret += 1
    if toplam > hedef:
        budanan += 1                        # torunların hepsi de aşar
        return
    if i == n:
        if toplam == hedef:
            cozum_isle()
        return
    budamali(i + 1, toplam)
    yol.append(a[i])
    budamali(i + 1, toplam + a[i])
    yol.pop()
```

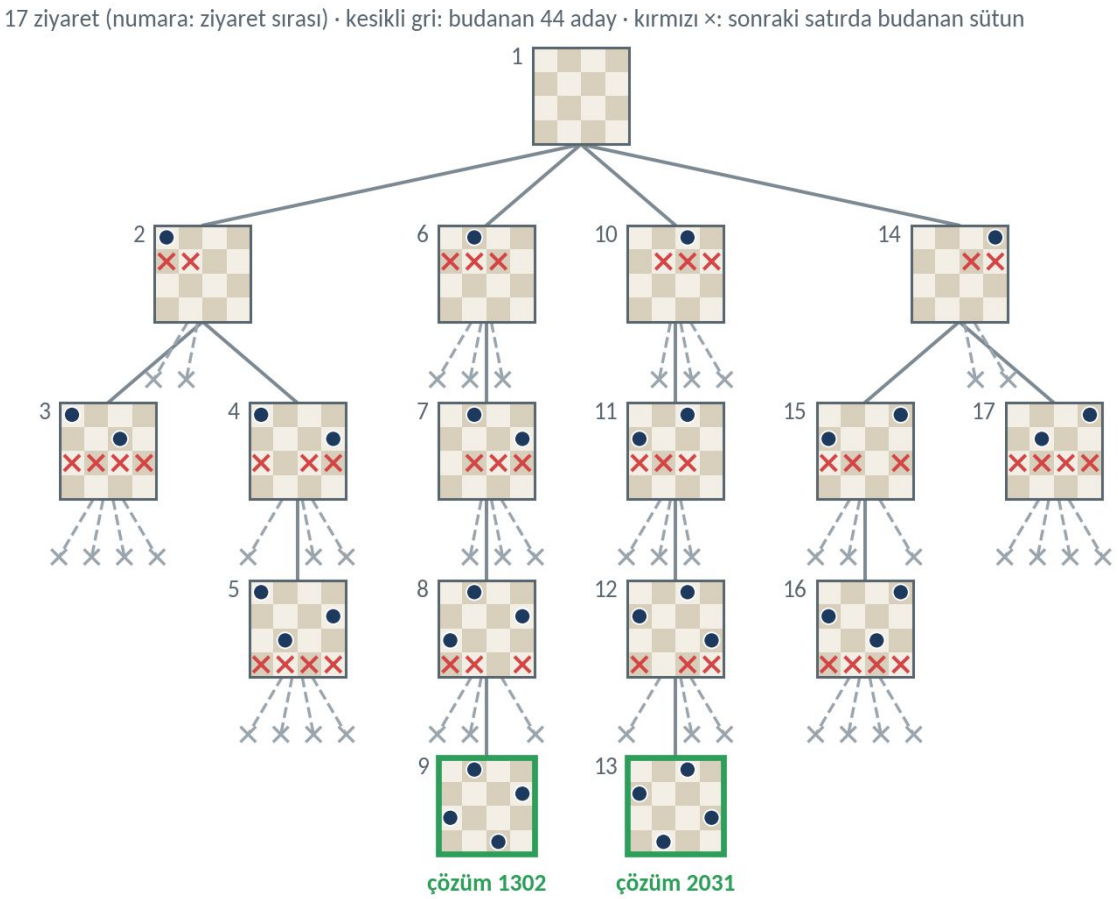
> DİL PENCERESİ · **Farklılaşan satırlar**
>
> Ortak LCG üç dilde aynı diziyi üretir: C'de `rastgele_dizi(a, n, 1, 40)` var olan diziyi doldurur, C++ ve Python'da yeni bir vektör/liste döndürür. Budama koşulu `toplam > hedef` üç dilde aynıdır ve yalnızca negatif olmayan sayılarda geçerlidir (Hata Avcısı 4). Seçim yığını C'de `yol[uzunluk++]` … `uzunluk--`, C++'ta `push_back`/`pop_back`, Python'da `append`/`pop` ile tutulur.

9 · Test. Üç yöntemin çözüm sayıları her girdide aynıdır (2, 37 ve 2 090); kaba kuvvet ötekilerin kâhinidir. **10 · Deney.** Mini-OBS 2 v2.2'de 30 derslik havuzda üst sınır budaması 1 073 741 824 düğümlük ağacın 1 434 525 düğümünü ziyaret eder. **11 · Varyantlar.** İlk çözümde durmak; küçük tamsayılarda O(n·hedef) dinamik programlama (H13); en iyi değer aranıyorsa dal ve sınır (branch and bound).

4.4 N-vezir (geri izleme)

1 · Problem. N × N tahtaya birbirini tehdit etmeyen N vezir yerleştirmenin bütün yolları. Her satırda tam bir vezir olacağından yerleşim `sutun[r]` dizisiyle tutulur; N = 4'ün çözümleri (1, 3, 0, 2) ve (2, 0, 3, 1). **2 · Fikir.** Satır satır ilerlenir; r. satırda her c sütunu soldan sağa denenir; önceki bir vezirle aynı sütun ya da köşegendeki aday budanır. Güvenli karede vezir konur (seç), r + 1. satıra geçilir (dene), dönünce kaldırılır (geri al). Bir satırda hiç güvenli kare yoksa bir önceki satıra dönülür.

Şekil 3.9
4-vezir durum uzayı ağacı: 17 ziyaret edilen düğüm (mini tahta; numara ziyaret sırası) ve 44 budanan aday (kesikli gri); iki çözüm yeşil çerçeveli



3 · Elle izleme. (0, 0) ve (1, 2) konduktan sonra 2. satırın dört sütunu da budanır, (1, 2) geri alınır; (1, 3)'ten sonra (2, 1) konur ama 3. satırda yine aday kalmaz ve (0, 0) dalının tamamı başarısız olur. Arama 17 düğüm ziyaret eder ve 44 aday budar (Şekil 3.9).

> TAHMİN ET · **Sizce bu sütun güvenli mi?**
>
> Vezirler (0, 1), (1, 3), (2, 0)'dayken 3. satırda hangi sütun güvenlidir? (Yanıt: yalnızca c = 2; c = 0, 1 ve 3 sütun kısıtıyla budanır. İlk çözüm (1, 3, 0, 2) bulunur.)

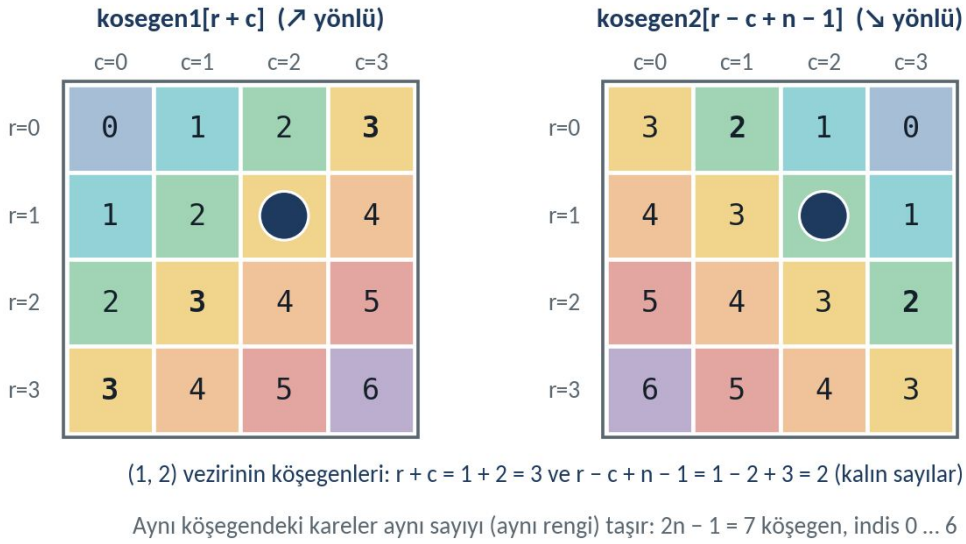
SÖZDE KOD · 4 · Sözde kod: sütun ve köşegen dizileriyle N-vezir

```
algoritma YERLEŞTİR(r)                  ▷ 0..r−1 satırlarında birer vezir var
    eğer r = n ise
        çözüm sayısını artır; döndür
    c ← 0'dan n−1'e kadar her c için
        eğer sütun[c], köşegen1[r+c] ve köşegen2[r−c+n−1] boş ise
            üçünü işaretle                  ▷ seç
            YERLEŞTİR(r + 1)                ▷ dene
            işaretleri kaldır               ▷ geri al
```

5 · Doğruluk. Değişmez (Tablo 3.4): yerlestir(r) çağrısında 0..r−1 satırlarındaki vezirler birbirini tehdit etmez ve sütun / köşegen dizileri tam olarak bu vezirleri işaretler. Satır başına bir vezir genelliği

kaybettirmez; çakışma sonradan ne konursa konsun sürdüğü için budama güvenlidir; her satırda bütün sütunlar denendiği için eksik çözüm kalmaz. **Köşegen indisleri:** "↗" köşegende r + c, "↘" köşegende r − c sabittir; n − 1 eklenerek 0 … 2n − 2'ye kaydırılır (Şekil 3.10). Böylece güvenlik denetimi önceki r veziri taramak (O(r)) yerine üç dizi okumasıdır (O(1)).

Şekil 3.10
4 × 4 tahtada köşegen indisleri kosegen1[r + c] ve kosegen2[r − c + n − 1]; aynı renk aynı köşegen



6 · Analiz. Sütun kısıtı r. satırda en çok n − r aday bırakır: ziyaret edilen düğüm sayısı ⌊e·n!⌋'i geçemez, yani O(n!) olur. Kimlik kartındaki O(n!) üst sınırı bu düğüm sayısıdır; her düğüm n sütunu O(1)'de denetlediği için süre için kaba üst sınır O(n·n!)'dir. İki sınır da gevşektir. Köşegen budamasıyla gerçek sayı çok küçüktür: N = 8'de 2 057 düğüm (8! = 40 320). Ek bellek O(n).

Tablo 3.4
N-vezir (geri izleme) kimlik kartı

ALGORİTMA KİMLİK KARTI **N-vezir (geri izleme)** *(N-queens (backtracking))*	
Strateji	Geri izleme (budamalı; satır başına bir vezir, sütun ve köşegen dizileriyle O(1) güvenlik denetimi)
Girdi önkoşulu	Yok (N ≥ 1); N = 2 ve N = 3 için çözüm yoktur
Zaman (en iyi · ortalama · en kötü)	— · — · O(n!) üst sınır
Ek bellek	O(n)
Özellikler	Budama, ziyaret edilen düğüm sayısını büyük ölçüde azaltır: N = 8'de budamasız ağacın 19 173 961 düğümüne karşı 2 057 düğüm (≈ 9 321 kat), 92 çözüm; N = 12'de 856 189 düğüm, 14 200 çözüm. Köşegen indisleri r + c ve r − c + n − 1
Değişmez / doğruluk	yerlestir(r) çağrısında 0..r−1 satırlarındaki vezirler birbirini tehdit etmez ve sütun / köşegen dizileri tam olarak bu vezirleri işaretler
Kütüphane karşılığı	C: yok · C++: yok · Python: yok (kâhin: `itertools.permutations(range(n))` ile n! sütun düzenini tek tek denemek)
Ne zaman kullanılır	Kısıt sağlama problemlerinde (yerleştirme, çizelgeleme, bulmacalar) tüm çözümler ya da bir çözüm gerektiğinde; tek çözüm yetiyorsa ilk çözümde durulur (Hata Avcısı 2)

Ne zaman kullanılmaz	N büyüdükçe tüm çözümleri saymak hızla uygulanamaz hâle gelir (çözüm sayısının kendisi üstel büyür); budama olmadan hiç kullanılmaz

7 · C gerçekleştirimi. Örnek 9'da `guvenli_mi` önceki r veziri tarar (O(r)); `yerlestir` ise vezir koyduktan sonra geri almayı açıkça yazmaz, çünkü `sutun[r]` kardeş dalda yeniden yazılır (Ek A.9):

c/H03_09_n_vezir.c · satır 21–30

```c
static int guvenli_mi(int r, int c)
{
    for (int i = 0; i < r; i++) {
        int fark = sutun[i] - c;
        if (fark == 0 || fark == r - i || fark == i - r) {
            return 0;              /* aynı sütun ya da aynı köşegen */
        }
    }
    return 1;
}
```

Örnek 10 aynı aramayı sütun ve köşegen dizileriyle yapar; geri alma açıktır:

c/H03_10_n_vezir_hizli.c · satır 55–83

```c
static int bos_mu(int r, int c)
{
    denetim_sayisi++;
    return !sutun_dolu[c] && !kosegen1[r + c]
           && !kosegen2[r - c + n - 1];
}

static void isaretle(int r, int c, int deger)
{
    sutun_dolu[c] = deger;
    kosegen1[r + c] = deger;
    kosegen2[r - c + n - 1] = deger;
}

static void yerlestir_dizi(int r)
{
    dugum_sayisi++;
    if (r == n) {
        cozum_sayisi++;
        return;
    }
    for (int c = 0; c < n; c++) {
        if (bos_mu(r, c)) {
            isaretle(r, c, 1); /* seç */
            yerlestir_dizi(r + 1); /* dene */
            isaretle(r, c, 0); /* geri al */
        }
    }
}
```

Program N = 4 … 10 için iki temsili yan yana çalıştırır: düğüm ve çözüm sayıları aynıdır; N = 8'de tarama 46 752 karşılaştırma, dizi 15 720 kare denetimi yapar. **8 · Üç dil karşılaştırması.** Arama fonksiyonu üç

dilde aynı yapıdadır; C++'ta işaret dizileri `std::vector<bool>`'dur (aşağıdaki bildirimler; her öğe tek bit, Bölüm 8), Python'da `True`/`False` listeleridir:

C++ **H03_10_n_vezir_hizli.cpp · satır 22–24**

```cpp
static std::vector<bool> sutun_dolu(MAKS_N);    // c sütununda vezir var mı?
static std::vector<bool> kosegen1(2 * MAKS_N - 1); // r + c sabit
static std::vector<bool> kosegen2(2 * MAKS_N - 1); // r - c + n - 1 sabit
```

Python **H03_10_n_vezir_hizli.py · satır 63–73**

```python
def yerlestir_dizi(r):
    global dugum_sayisi, cozum_sayisi
    dugum_sayisi += 1
    if r == n:
        cozum_sayisi += 1
        return
    for c in range(n):
        if bos_mu(r, c):
            isaretle(r, c, True)    # seç
            yerlestir_dizi(r + 1)   # dene
            isaretle(r, c, False)   # geri al
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Ölçülecek iş C'de fonksiyon işaretçisi ve `void *` veriyle, C++'ta lambda (`[&] { … }`), Python'da `lambda:` ile `olc_ortanca`'ya verilir. C++'taki `std::vector<bool>` her öğeyi bir bitte saklayan özel bir kaptır. Süreler stderr'e yazıldığından stdout üç dilde aynıdır.

9 · Test. Örnek 9 N = 1 … 10 için çözüm sayılarını bilinen değerlerle (1, 0, 0, 2, 10, 4, 40, 92, 352, 724) karşılaştırır; Örnek 10 iki temsilin sayaçlarını karşılaştırır (“FARK” yok); N = 2 ve 3 çözümsüzdür. **10 · Deney.** Bölüm 6: N = 8'de budamasız 19 173 961, budamalı 2 057 düğüm. **11 · Varyantlar.** İlk çözümde durmak (114 düğüm; Hata Avcısı 2), simetri (Alıştırma 3.12), bit maskeleri (Merak Kutusu); aynı şablon labirenti ve Sudoku'yu çözer (Bölüm 5).

5. Çözümlü Örnekler

Örnek merdiveninin her basamağı bir öncekine yalnızca bir yeni öge ekler. On üç basamağın sekizi Bölüm 3–4'te, Örnek 12 Bölüm 8'de işlenir; bu bölüm kalan dört basamağı çözer: bit maskesi, k-kombinasyonlar, labirent ve Sudoku (Tablo 3.5). Bütün örneklerin C++ ve Python karşılıkları Ek A'dadır; dosya dizini `kodlar/KOD_DIZINI.md`'dedir.

Tablo 3.5
Örnek merdiveni ve ders notundaki yerleri

#	Örnek (kodlar/c, cpp, python)	Yeni öge	Düzey	Çıktı	Bölüm
1	H03_01_ozyineleme_sayac	Ağaç ↔ yığın ilişkisi	★	=	3.1
2	H03_02_hanoi	Yerine koyma yöntemi	★	=	3.2
3	H03_03_ikili_diziler	Dahil et/etme ağacı	★	=	3.3
4	H03_04_alt_kumeler	Üretme şablonu	★★	=	4.1
5	H03_05_bit_maskesi	Özyinelemesiz üretim	★★	=	5.1

#	Örnek (kodlar/c, cpp, python)	Yeni öge	Düzey	Çıktı	Bölüm
6	H03_06_permutasyonlar	Seç–dene–geri al	★★	=	4.2
7	H03_07_kombinasyonlar	C(n, k) sayımı	★★	=	5.2
8	H03_08_alt_kume_toplami	Budama	★★	=	4.3
9	H03_09_n_vezir	Durum uzayında arama	★★★	=	4.4
10	H03_10_n_vezir_hizli	Durum temsilinin etkisi	★★★	=	4.4
11	H03_11_labirent	Izgara durum uzayı (H11'e köprü)	★★	=	5.3
12	H03_12_ozyineleme_siniri	Dil Penceresi: yığın sınırları	★★	Δ	8
13	H03_13_sudoku	Kısıt yayılımı olmadan geri izleme	★★★	=	5.4

5.1 Bit maskesiyle alt kümeler: özyinelemesiz üretim (Örnek 5)

n elemanlı bir kümenin alt kümeleri, 0 … 2^n − 1 arasındaki tamsayılarla birebir eşlenir: maskenin k. biti 1 ise a[k] kümededir (AP-1 H4'teki bit operatörleri). Böylece özyineleme ya da yığın olmadan, tek bir döngüyle bütün aile dolaşılır. Hangi bitin hangi elemana karşılık geldiği bir sözleşmedir: a[0] en soldaki bite (b_{n-1}) eşlenirse sıra Örnek 4'ün dahil et/etme sırasıyla aynıdır; en sağdaki bite (b_0) eşlenirse aile aynı, sıra farklıdır (Şekil 3.11).

c/H03_05_bit_maskesi.c · satır 41–54

```c
static void maske_ile_uret(int n)
{
    char m1[64], m2[64];
    printf("maske  ikili  a[0] soldaki bit  a[0] sağdaki bit\n");
    for (unsigned maske = 0; maske < (1u << n); maske++) {
        printf("%5u  ", maske);
        for (int b = n - 1; b >= 0; b--) {
            putchar((maske >> b) & 1u ? '1' : '0');
        }
        kume_metni(maske, n, 1, m1);
        kume_metni(maske, n, 0, m2);
        printf("    %-16s  %s\n", m1, m2);
    }
}
```

Şekil 3.11
n = 3 için maske ↔ alt küme eşlemesi: a[0] soldaki bit (Örnek 4'ün sırası) ve a[0] sağdaki bit (aynı aile, farklı sıra)

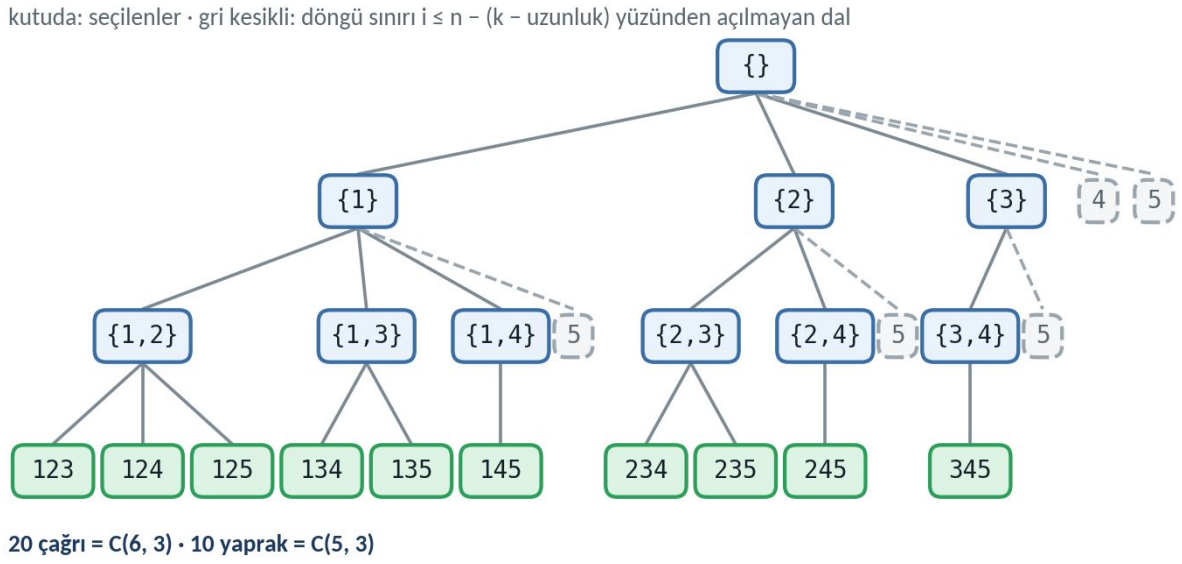
maske	ikili $b_2b_1b_0$	a[0] ↔ soldaki bit (b_2) = Örnek 4'ün sırası	a[0] ↔ sağdaki bit (b_0) aynı aile, farklı sıra
0	000	{}	{}
1	001	{3}	{1}
2	010	{2}	{2}
3	011	{2, 3}	{1, 2}
4	100	{1}	{3}
5	101	{1, 3}	{1, 3}
6	110	{1, 2}	{2, 3}
7	111	{1, 2, 3}	{1, 2, 3}

Döngü sınırı `1u << n` işaretsiz yazılır: `1 << n` bir `int` ifadesidir ve n ≥ 31'de tanımsız davranıştır (Hata Avcısı 6). Program n = 1 … 16 için bit maskesiyle ürettiği aileyi Örnek 4'ün özyinelemeli sırasıyla karşılaştırır ve her kümenin tam bir kez çıktığını denetler: yavaş ama apaçık doğru bir yöntem, hızlı ya da karmaşık olanın **kâhinidir**. Zaman yine Θ(n·2ⁿ)'dir: her maske için n bit okunur.

5.2 k-kombinasyonlar: başlangıç indisi (Örnek 7)

{1, …, n} içinden k elemanlı bütün kümeleri istiyoruz. Alt küme ağacının yalnızca k elemanlı yapraklarını yazdırmak 2^n yaprağın çoğunu boşa üretir. Başlangıç indisi şablonunda `kombinasyon(bas, k)` yalnızca a[bas..n − 1] arasından seçer ve seçtiği a[i]'den sonra `kombinasyon(i + 1, k)` çağırır; indisler hep arttığı için {2, 1} gibi ikinci bir sıra hiç oluşmaz.

Şekil 3.12
n = 5, k = 3 için başlangıç indisli ağaç: 20 çağrı, 10 yaprak; kesikli gri dallar döngü sınırı yüzünden açılmaz



c/H03_07_kombinasyonlar.c · satır 35–49

```c
static void kombinasyon(int bas, int k)
{
    cagri_sayisi++;
    if (uzunluk == k) {            /* k eleman seçildi */
        komb_isle();
        return;
    }
    /* i en çok n - (k - uzunluk): sağda yeterli eleman kalmalı */
    for (int i = bas; i <= n - (k - uzunluk); i++) {
        yol[uzunluk] = a[i];       /* seç */
        uzunluk++;
        kombinasyon(i + 1, k);     /* dene: yalnızca i'den sonrakiler */
        uzunluk--;                 /* geri al */
    }
}
```

Döngünün üst sınırı `n − (k − uzunluk)` bir budamadır: sağda yeterli eleman kalmayan dallar açılmaz (Şekil 3.12, kesikli gri). n = 5, k = 3 için ağaçta 20 çağrı ve 10 yaprak vardır; programın tablosu çağrı sayısının her satırda C(n + 1, k) olduğunu gösterir (n = 20, k = 10: 352 716 = C(21, 10); kanıtı Merak Kutusu'nda). C(n, k) ayrıca çarpımsal formülle hesaplanır: `c = c * (m − k + i) / i` her adımda tam bölünür, çünkü ara değer C(m − k + i, i) bir tamsayıdır.

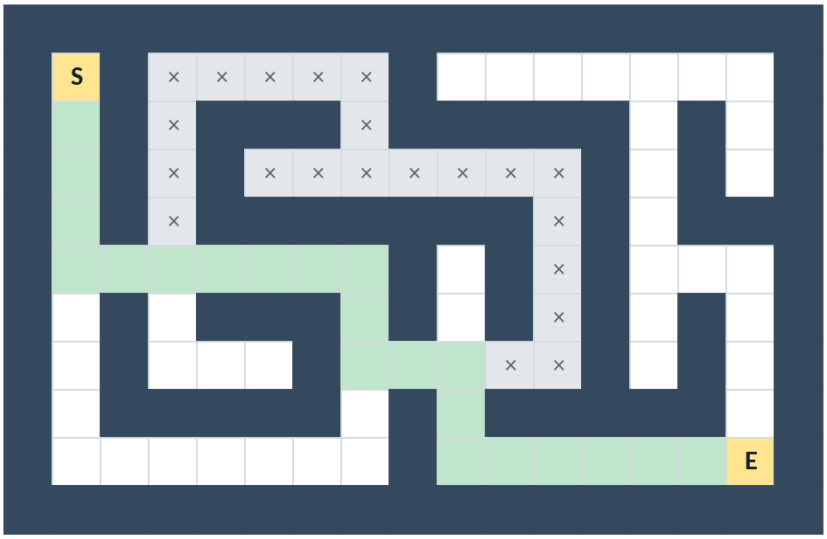
5.3 Labirentte yol bulma: ızgarada geri izleme (Örnek 11)

Labirent stdin'den okunur (`girdiler/H03_11_labirent.txt`, 11 × 17; `#` duvar, `.` boş, `S` giriş, `E` çıkış). Durum bir hücredir; dört komşu sabit sırayla denenir: sağ, aşağı, sol, yukarı. Girilen hücre `*` ile işaretlenir (seç); hiçbir yön çıkışa götürmezse `x` yapılır (geri al). İşaretler döngüleri de önler.

c/H03_11_labirent.c · satır 32–52

```c
static int yol_bul(int r, int c)
{
    ziyaret++;
    if (izgara[r][c] == 'E') {
        return 1;                      /* çıkışa ulaşıldı */
    }
    if (izgara[r][c] != 'S') {
        izgara[r][c] = '*';            /* seç: hücre yola eklenir */
    }
    for (int d = 0; d < 4; d++) {
        int yr = r + DR[d], yc = c + DC[d];
        if (gecilebilir(yr, yc) && yol_bul(yr, yc)) {
            return 1;                  /* yol bulundu: '*'lar kalır */
        }
    }
    if (izgara[r][c] != 'S') {
        izgara[r][c] = 'x';            /* geri al: buradan çıkış yok */
    }
    geri_donulen++;
    return 0;
}
```

Şekil 3.13
Örnek 11'in son ızgarası: yeşil yol, gri × geri dönülen hücreler



Program 44 hücreye girer ve 21 hücreden geri döner (Şekil 3.13). Bulunan yol 22 adımdır ama **en kısa olduğu güvence edilmez**: derinlik öncelikli arama ilk bulduğu yolu döndürür ve bu yön sırasına bağlıdır. En kısa yolu genişlik öncelikli arama (BFS) bulur (H11). Hatalı girdide program üç dilde de 1 çıkış koduyla durur.

5.4 Sudoku çözücüsü: kısıt yayılımı olmadan geri izleme (Örnek 13)

Meydan okuma basamağı N-vezirin dizili denetimini 9 × 9'a taşır: her satır, sütun ve 3 × 3 kutu için "d rakamı kullanıldı mı?" dizileri (`satir_var`, `sutun_var`, `kutu_var`) tutulur ve uygunluk O(1)'de denetlenir. `coz` satır öncelikli ilk boş kareyi bulur, 1'den 9'a uygun her rakamı dener ve **ilk çözümde durur**: dönüş değeri 1 yukarı iletilir (tek çözüm yeter; Hata Avcısı 2'nin öbür yüzü). Gerçek çözücülerin kullandığı kısıt yayılımı (constraint propagation), yani her yerleştirmeden sonra öteki karelerin seçeneklerini daraltma, burada bilerek yoktur; C++ ve Python kodu Ek A.13'tedir.

c/H03_13_sudoku.c · satır 43–63

```c
static int coz(int k)
{
    dugum_sayisi++;
    while (k < 81 && tablo[k / 9][k % 9] != 0) {
        k++;                        /* dolu kareleri atla */
    }
    if (k == 81) {
        return 1;                   /* boş kare kalmadı: çözüldü */
    }
    int r = k / 9, c = k % 9;
    for (int d = 1; d <= 9; d++) {
        if (uygun_mu(r, c, d)) {
            koy(r, c, d, 1);        /* seç */
            if (coz(k + 1)) {       /* dene */
                return 1;           /* tek çözüm yeter: dur */
            }
            koy(r, c, d, 0);        /* geri al */
```

```
60          }
61      }
62      return 0;                       /* hiçbir rakam olmadı: geri dön */
63  }
```

Program 456 082 düğüm ziyaret eder ve çözümü `dogrula` ile denetler (her satır, sütun ve kutuda 1–9 bir kez; ipuçları korunmuş): `doğrulama: geçerli`. Bu makinede çözme süresi C'de ≈ 22 ms, Python'da ≈ 0,77 s'dir (stderr, tek çalıştırma). Bulmaca bir tam ızgaradan, tek çözümü korunarak ipucu silinerek üretilmiştir; Atölye 3'te "en az seçenekli kare önce" sezgisi eklenir.

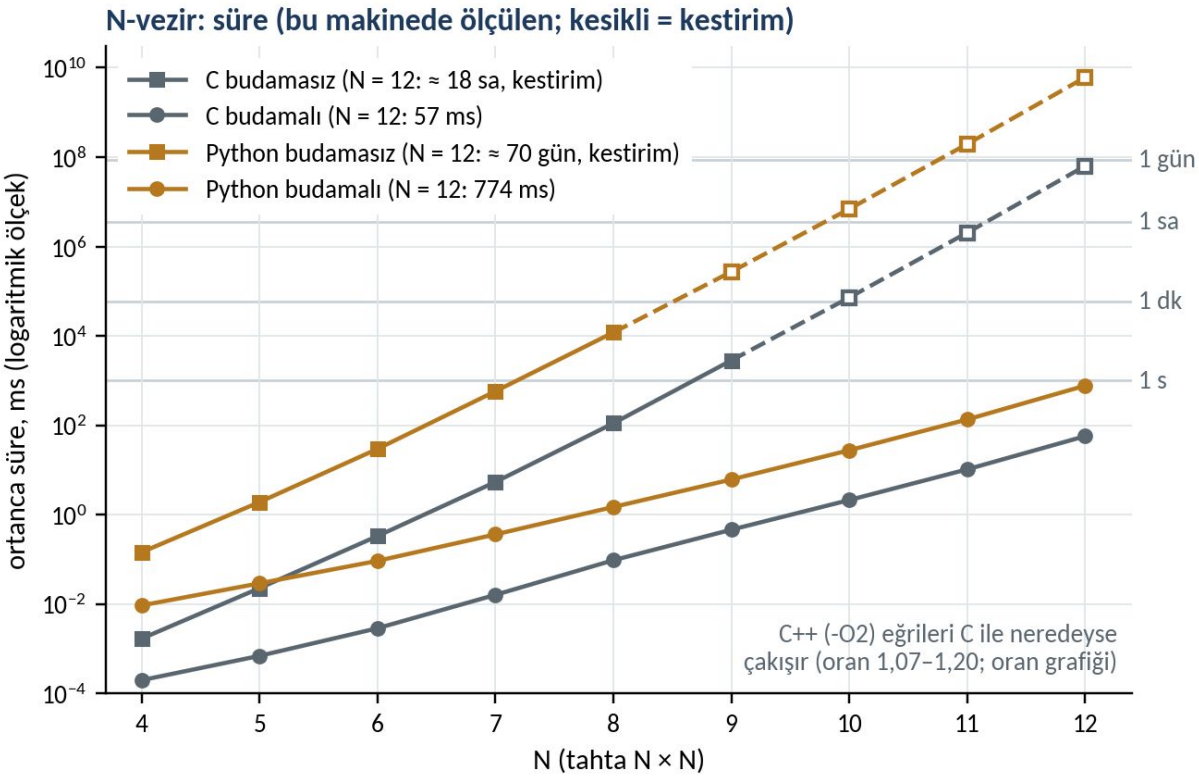
6. Haftanın Deneyi: N-Vezirde Budamanın Etkisi

Soru. Budama ziyaret edilen düğüm sayısını ve süreyi ne kadar azaltır; bu kazanç dilin (C, C++, Python) getirdiği farkla nasıl karşılaştırılır? **Düzen.** İki program aynı arama ağacını farklı derinlikte dolaşır: `budamasiz` her satırda her sütunu dener ve yalnızca yaprakta bütün vezir çiftlerini denetler (ağaçta $1 + N + N^2 + \ldots + N^N$ düğüm); `budamali` Örnek 10'daki dizilerle her düğümde O(1) denetim yapar. Sürücü `kodlar/deney/olcum_calistir.py` C ve C++'ı -O2 ile (C'yi ayrıca -O0 ile) derler, her ölçümde 1 ısınma + 5 tekrar (büyük N'de 3) çalıştırıp ortancayı kaydeder (Sedgewick & Wayne, 2011). Budamasız arama C/C++'ta $N \le 8$, Python'da $N \le 7$ ölçüldü; kestirimi sınamak için C N = 9 ve Python N = 8 birer kez ölçüldü. Ortam (`deney/OLCUM_ORTAMI.txt`): Intel Xeon @ 2,80 GHz, 2 çekirdekli paylaşımlı sanal makine, gcc/g++ 13.3.0, Python 3.11.15.

Tablo 3.6
*Ölçülen ortanca süreler ve düğüm sayıları (h03_deney_olcum.csv, h03_deney_uzay.csv); * kestirim (h03_deney_kestirim.csv), [1] tek tekrarlı doğrulama ölçümü*

N	budamasız düğüm	budamalı düğüm	çözüm	C -O2 budamasız	C -O2 budamalı	Python budamasız	Python budamalı
4	341	17	2	1,7 µs	0,2 µs	0,145 ms	9,4 µs
5	3 906	54	10	22,9 µs	0,7 µs	1,93 ms	29,3 µs
6	55 987	153	4	0,335 ms	2,9 µs	29,8 ms	93,2 µs
7	960 800	552	40	5,42 ms	16,2 µs	579,6 ms	0,364 ms
8	19 173 961	2 057	92	111,6 ms	96,3 µs	11,98 s[1]	1,50 ms
9	435 848 050	8 394	352	2,84 s[1]	0,464 ms	≈ 4,5 dk*	6,16 ms
10	11 111 111 111	35 539	724	≈ 1,2 dk*	2,14 ms	≈ 1,9 sa*	27,8 ms
11	313 842 837 672	166 926	2 680	≈ 34,1 dk*	10,4 ms	≈ 2,3 gün*	135,6 ms
12	9 726 655 034 461	856 189	14 200	≈ 17,6 sa*	57,4 ms	≈ 70,4 gün*	774,0 ms

Şekil 3.14
Ortanca süreler (y logaritmik); kesikli ve içi boş işaretler kestirimdir



Kestirim. Ölçülemeyen boyutlar için "düğüm başına süre sabittir" modelini kullandık: $t(N) = t(N_0) \cdot \text{düğüm}(N)/\text{düğüm}(N_0)$, N_0 o seride ölçülen en büyük N. Model iki doğrulama ölçümüyle sınandı: C -O2 N = 9 için kestirim 2,54 s, ölçülen 2,84 s (%10,6 düşük); Python N = 8 için kestirim 11,57 s, ölçülen 11,98 s (%3,5 düşük). Model büyük N'de hafifçe iyimserdir (yaprak denetimi N ile pahalılaşır) ama büyüklük sırası için yeterlidir: tablodaki * işaretli değerler bu modelden gelir (ör. N = 12 budamasız arama C'de ≈ 17,6 sa sürerdi; budamalı arama 57,4 ms sürer).

Kuramla karşılaştırma ve yorum.

- Budamasız düğüm sayısı formüldür; N'den N + 1'e geçerken yaklaşık $(N + 1)\cdot(1 + 1/N)^N$ kat, yani $e\cdot(N + 1)$'e yakın büyür (8 → 9: 22,7 kat). C'de ölçülen süre oranı aynı düzendedir (8 → 9: 25,4). Budamalı düğüm sayısı N başına yalnızca 3–5 kat büyür (11 → 12: 5,13); süre oranı biraz daha büyüktür (5,52), çünkü düğüm başına N sütun denetlenir.
- **Budamanın etkisi dil farkını gölgede bırakır.** N = 8'de budama düğüm sayısını 9 321 kat, N = 12'de 11,4 milyon kat azaltır; Python ise C'den yalnızca 13–107 kat yavaştır. Python'daki budamalı program (N = 8: 1,50 ms) C'deki budamasız programdan (111,6 ms) hızlıdır.
- Python/C oranı yönteme göre değişir: budamasızda ≈ 107×, budamalıda ≈ 13–16× (h03_deney_oran.csv). Olası açıklama (ölçülmedi, yorumdur): C'de budamasız düğüm çok ucuzdur (≈ 5,8 ns), budamalı düğüm N sütunun denetimini ve öngörülmesi güç dallanmaları içerir (≈ 67 ns); Python'da ikisinin maliyetini de yorumlayıcı yükü belirler (≈ 625 ve 904 ns).
- C++ C'ye çok yakındır (oran 1,07–1,20); -O0, C'yi budamasız aramada 1,9–2,7, budamalıda ≈ 1,43–1,49 kat yavaşlatır.

Sınırlılıklar. Tek paylaşımlı sanal makine; N ≤ 7 budamalı süreleri 50 µs'nin altındadır (oran tablosuna alınmadı); doğrulama ölçümleri tek tekrarlıdır; kestirimler modeldir; yalnızca Python 3.11. **Kendi makinenizde** `kodlar/deney/` altında `python3 olcum_calistir.py` ile tekrarlayın: düğüm sayıları aynı, süreler farklı çıkacaktır.

7. Hata Avcısı

Yedi hatanın her biri `kodlar/hata_avcisi/` altında kasıtlı hatalı ve düzeltilmiş dosya çifti olarak bulunur; belirtiler bu makinede gözlenen çıktılardır (`hata_avcisi/ciktilar/`). Hiçbiri derleyici uyarısı üretmez.

Tablo 3.7
Hata Avcısı: hata, belirti, neden ve çözüm

#	Hata	Belirti (bu makinede)	Neden	Çözüm
1	Seçimi geri almayı unutmak (durum kirlenmesi)	6 yerine 1 permütasyon (123); sonda kullanildi = {1, 1, 1}	İlk yaprağa inerken konan işaretler kalkmıyor; kardeş dallar elemanları kullanılmış görüyor	Her "seç"in simetrik "geri al"ı: kullanildi[i] = 0
2	Tüm çözümler istenirken ilk çözümde dönmek	N = 8: 92 yerine 1 çözüm (114 düğüm)	"bulundu → 1 döndür" kalıbı sayma fonksiyonunda	İki sözleşme, iki fonksiyon: tum_cozumler / ilk_cozum
3	Tekrar eden elemanlarda yinelenen permütasyonlar	(1, 1, 2): 3 yerine 6 sonuç	İki 1 farklı indisli olduğu için ayrı eleman sayılıyor	Sıralı girdi + "eşit eleman, soldaki kullanılmadıysa atla"
4	"Toplam aşıldı" budamasını negatif sayıda uygulamak	a = {5, 4, −3, 2}, hedef 6: 2 yerine 1 çözüm	"toplam > hedef ⇒ torunlar da aşar" önermesi negatif elemanla yanlış	Budamayı yalnızca a[i] ≥ 0 iken kullan
5	Eksik temel durum	Python: RecursionError; C korumasız -O0: SIGSEGV (139); -O2: 10 s'de bitmedi	Adım 2 ile tek n hiç 0 olmaz (7, 5, 3, 1, −1, …)	Temel durum `n <= 0`
6	`1 << n`'in n ≥ 31'de taşması	n = 31: −2147483648, n = 32: 1; derleyici uyarısı yok; UBSan bildirir	`1` bir `int`'tir; iki durum da tanımsız davranış	`1ULL << n`, `uint64_t`
7	Paylaşılan durumun kopyalanmaması (Python)	sonuc = [[], [], …, []], aynı nesne mi? True	`append(yol)` başvuruyu ekler; yol sonra boşalır	`sonuc.append(yol[:])`

HA1 · Geri almayı unutmak.

C `hata_avcisi/H03_HA1_geri_alma_unutma.c · satır 25–31`

```c
    for (int i = 0; i < N; i++) {
        if (!kullanildi[i]) {
            kullanildi[i] = 1;      /* seç */
            yol[k] = a[i];
            permutasyon(k + 1);     /* dene */
                                    /* HATA: geri al unutuldu */
        }
```

GÖZLENEN

```
$ ./H03_HA1_geri_alma_unutma
123
permütasyon sayısı = 1 (beklenen 6)
kullanildi = {1, 1, 1}
(çıkış kodu: 0)
```

İlk yaprağa (123) inen yolun işaretleri hiç kalkmaz; kardeş dallar her elemanı kullanılmış görür. Düzeltme 30. satıra `kullanildi[i] = 0;` eklemektir (6 permütasyon, sonda {0, 0, 0}). Geri izlemede **her seçimin bir eşi olmalıdır**.

HA2 · İlk çözümde dönmek / ağacın tamamını dolaşmak.

`hata_avcisi/H03_HA2_ilk_cozumde_donmek.c` bütün çözümleri saymak isterken `if (yerlestir(r + 1)) return 1;` yazar ve şunu bildirir: `N = 8: çözüm sayısı = 1 (beklenen 92); ziyaret edilen düğüm = 114`.

Sudoku'da doğru olan "ilk çözümde dur" kalıbı, bütün çözümleri sayan bir fonksiyonda aramayı keser; ters hata, tek çözüm yeterken ağacın tamamını dolaşmaktır. Düzeltilmiş dosya iki sözleşmeyi iki fonksiyona ayırır: `tum_cozumler` 2 057, `ilk_cozum` 114 düğüm ziyaret eder (Bölüm 9).

HA3 · Tekrar eden elemanlar (C++).

`hata_avcisi/H03_HA3_tekrarli_permutasyon.cpp` Örnek 6'nın kullanildi yöntemini {1, 1, 2}'ye uygular ve 112 121 112 121 211 211 yazar (6 sonuç, 3 farklı). Düzeltilmiş sürümde girdi sıralıdır ve `kullanildi[i] || (i > 0 && a[i] == a[i - 1] && !kullanildi[i - 1])` ise aday atlanır: eşit elemanlar yalnızca soldan sağa kullanılır ve her diziliş bir kez çıkar (112, 121, 211; `std::next_permutation` da böyledir, Kavram 2).

HA4 · Negatif sayıda "toplam aşıldı" budaması.

C `hata_avcisi/H03_HA4_negatif_budama.c · satır 30–34`

```c
static void budamali(int i, int toplam)
{
    if (toplam > hedef) {
        return;                    /* HATA: -3 eklenince toplam azalabilir */
    }
```

Program `bulunan çözüm = 1 (beklenen 2)` yazar: {5, 4} düğümünde 9 > 6 olduğu için dal kesilir, oysa −3 eklenince 6'ya inilir. Kanıt "eklenecek her sayı ≥ 0" varsayımına dayanıyordu; düzeltilmiş sürüm negatif eleman varsa budamayı kapatır.

HA5 · Eksik temel durum.

`ikiser_toplam` ikişer azalan toplamı hesaplar, ama yalnızca `n == 0`'da durur; tek n için (7, 5, 3, 1, −1, …) 0 hiç gelmez:

Python `hata_avcisi/H03_HA5_eksik_temel_durum.py · satır 10–14`

```python
def ikiser_toplam(n):
    """n + (n - 2) + (n - 4) + ... (ikişer azalan toplam)"""
    if n == 0:                 # HATA: tek n için 0 atlanır (n <= 0 olmalı)
        return 0
    return n + ikiser_toplam(n - 2)
```

GÖZLENEN · PYTHON

```
ikiser_toplam(6) = 12
Traceback (most recent call last):
…
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
(çıkış kodu: 1)
```

C'de aynı hata **sessizdir**: `H03_HA5_eksik_temel_durum.c` varsayılan çalıştırmada derinliği 100 000'de keserek durur; `korumasiz` argümanıyla -O0'da SIGSEGV ile sonlanır (çıkış kodu 139), -O2'de GCC özyinelemeyi döngüye çevirdiği için 10 saniyede bitmez: aynı hata, iki farklı belirti.

HA6 · Bit kaydırmada taşma (1 << n).

C `hata_avcisi/H03_HA6_bit_kaydirma.c · satır 16–19`

```c
static long long alt_kume_sayisi(int n)
{
    return 1 << n;                /* HATA: int'te 1 << 31 ve 1 << 32 */
}
```

GÖZLENEN (SON SATIR -FSANİTİZE=UNDEFİNED İLE)

```
n = 30: 2^n = 1073741824
n = 31: 2^n = -2147483648
n = 32: 2^n = 1
n = 33: 2^n = 2
H03_HA6_bit_kaydirma.c:18:14: runtime error: left shift of 1 by 31 places cannot be
represented in type 'int'
```

Program alt küme sayısını `1 << n` ile hesaplar ve n = 31, 32, 33 için -2147483648, 1 ve 2 yazar. `1` bir `int` olduğundan `1 << 31` sonucu `int`'e sığmaz, `1 << 32`'de kaydırma miktarı türün genişliğine ulaşır; ikisi de **tanımsız davranıştır** (undefined behavior; ISO/IEC, 2018, 6.5.7) ve başka derleyicide ya da eniyileme düzeyinde başka değerler çıkabilir. Değişken bir n için derleyici uyarmaz; UBSan (`-fsanitize=undefined`) çalışma anında bildirir. Düzeltme `1ULL << n`'dir (n ≤ 63); n = 33'te maske döngüsü doğru yazılsa bile 2^{33} ≈ 8,6 milyar adım sürer.

HA7 · Paylaşılan durumun kopyalanmaması (Python).

Python `hata_avcisi/H03_HA7_liste_kopyasi.py · satır 15–22`

```python
def alt_kumeler(i):
    if i == len(a):
        sonuc.append(yol)          # HATA: kopya değil, aynı liste
        return
    alt_kumeler(i + 1)
    yol.append(a[i])
    alt_kumeler(i + 1)
    yol.pop()
```

Program `sonuc = [[], [], [], [], [], [], [], []]` ve `aynı nesne mi? True` yazar: `sonuc.append(yol)` kopya değil, aynı liste nesnesine bir başvuru ekler; geri izleme `yol`'u boşalttıkça sekiz başvuru aynı boş listeyi gösterir. Düzeltme `sonuc.append(yol[:])`. C++'ta `push_back(yol)` zaten kopyalar: benzer sözdizimi, farklı anlam (Tshukudu & Cutts, 2020).

8. Dil Penceresi: C • C++ • Python

Tablo 3.8
Bu haftanın dil farkları

Öğe	C	C++	Python
Özyineleme sınırı	dilde yok; yığın boyutu (Linux: ulimit -s, tipik 8 MB)	C ile aynı	sys.getrecursionlimit() = 1000

Öğe	C	C++	Python
Sınır aşılınca	dil uyarmaz; Linux'ta tipik olarak SIGSEGV	C ile aynı	RecursionError (yakalanabilir)
Sınırı değiştirmek	ulimit -s; bağlayıcı (MinGW: -Wl,--stack)	C ile aynı	sys.setrecursionlimit(n) (güvenli değil)
Üreteç (generator)	yok: geri çağırma ya da açık yığın	C++17'de yok	yield, yield from
Permütasyon (kütüphane)	yok	std::next_permutation	itertools.permutations
Alt küme / kombinasyon	yok	yok	itertools.product / combinations, math.comb
Bit dizisi	int dizisi, unsigned maske	std::vector<bool> (vekil), std::bitset	bool listesi; int maske (sınırsız)
1 << n	int: n ≥ 31 tanımsız; 1u / 1ULL	C ile aynı	her n için doğru (sınırsız int)
Çözümü saklamak	diziyi elle kopyala	push_back(yol) kopyalar	append(yol) başvuru; yol[:] kopya
Takas	takas(&x, &y)	std::swap(x, y)	x, y = y, x
Modül düzeyi sayaç	static değişken	static değişken	global bildirimi gerekir

Özyineleme sınırı: Örnek 12 (Δ). Aynı `toplam(n)` üç dilde 900, 1000 ve 10 000 derinlikte çalıştırılır. C ve C++ üçünü de hesaplar; Python 1000 ve 10 000'de RecursionError yakalar. Program bilerek **-O0** ile derlenir: -O2'de GCC bu özyinelemeyi döngüye çevirdiği için (Bölüm 3.6, Kaputun Altında) yığın taşması (stack overflow) gözlenemez.

C `c/H03_12_ozyineleme_siniri.c · satır 17–23`

```c
static long long toplam(long long n)
{
    if (n == 0) {
        return 0;
    }
    return n + toplam(n - 1);
}
```

ÇIKTI · C

```
$ gcc -std=c17 -Wall -Wextra -pedantic -O0 H03_12_ozyineleme_siniri.c -o
H03_12_ozyineleme_siniri $ ./H03_12_ozyineleme_siniri
Özyineleme sınırı deneyi — C
toplam(n) = n + toplam(n - 1), toplam(0) = 0
dilin koyduğu derinlik sınırı: yok
derinlik    900: toplam = 405450
derinlik   1000: toplam = 500500
derinlik  10000: toplam = 50005000
```

ÇIKTI · PYTHON

```
$ python3 H03_12_ozyineleme_siniri.py
Özyineleme sınırı deneyi — Python
toplam(n) = n + toplam(n - 1), toplam(0) = 0
sys.getrecursionlimit() = 1000
derinlik    900: toplam = 405450
derinlik   1000: RecursionError yakalandı
derinlik  10000: RecursionError yakalandı
Python'da sınır yorumlayıcının koruma sayacıdır; aşılınca
RecursionError fırlatılır ve program yakalayıp sürdürebilir.
```

İsteğe bağlı argümanla tek bir derinlik denenebilir. Bu makinede (`ulimit -s` = 8192 KB) `./H03_12_ozyineleme_siniri 1000000` -O0 ile **Segmentation fault** verir (çıkış 139; çöken en küçük n ≈ 261 200); Python 3.11–3.13 sınırı n + 100'e yükselterek n = 1 000 000'u ≈ 0,4 s'de hesaplar. Python 3.10 ise aynı komutta n ≈ 18 000'in üstünde Segmentation fault ile çöktü: 3.10'da her Python çağrısı yorumlayıcının C yığınında da yer kaplar, 3.11'den beri Python'dan Python'a çağrıların çoğu C yığınını kullanmaz (Python Software Foundation, t.y.-b). `sys.setrecursionlimit` bir korumayı kaldırır, sorunu çözmez ve sonucu sürüme bağlıdır; derin özyineleme gerekiyorsa açık yığın kullanın (Kavram 4).

Üreteçlerle üretim (Python). `yield` kullanan bir fonksiyon çağrıldığında gövdesi çalışmaz, bir **üreteç** (generator) nesnesi döner; her `next` isteğinde fonksiyon kaldığı yerden bir sonraki `yield`'e kadar çalışır ve özyinelemeli üreticiler `yield from` ile bağlanır. Böylece alt kümeler **tembel** (lazy) üretilir: n = 30 için yalnızca ilk dört küme istenirse geri kalanı hiç üretilmez. Kavram 1 (algoritma Örnek 4'ün aynısı) sonuçları `itertools` kâhiniyle de karşılaştırır:

Python **kavram/H03_K01_uretec_alt_kume.py · satır 12–22**

```python
def alt_kumeler_uret(a, i=0, yol=None):
    """a[i..] için karar ver; her yaprakta yol'un kopyasını ver"""
    if yol is None:
        yol = []
    if i == len(a):
        yield tuple(yol)              # kopya: yol değişmeye devam eder
        return
    yield from alt_kumeler_uret(a, i + 1, yol)    # a[i] yok
    yol.append(a[i])                               # seç
    yield from alt_kumeler_uret(a, i + 1, yol)    # dene
    yol.pop()                                      # geri al
```

Program önce `{} {3} {2} {2, 3} {1} {1, 3} {1, 2} {1, 2, 3}` yazar; ardından n = 30 için yalnızca ilk dört kümeyi ister ve kâhinle karşılaştırır. `yield tuple(yol)` satırına dikkat: üreteç `yol`'un kendisini verseydi Hata Avcısı 7'nin tuzağı geri gelirdi. C ve C++17'de "duraklayıp sürme"nin doğrudan karşılığı yoktur; aynı etki bir geri çağırma fonksiyonu (fonksiyon işaretçisi ya da lambda) ya da açık bir yığınla (Kavram 4) elde edilir.

C++'ta vector<bool>. Standart `vector<bool>`'u her öğeyi bir bitte saklayan özel bir kap olarak tanımlar (ISO/IEC, 2017): `v[i]` bir `bool&` değil, o bite bağlı bir **vekil** (proxy) nesne döndürür. Kavram 5'te `auto y = v[0]; y = true;` bir kopyayı değil vektörün kendisini değiştirir; `vector<int>`'te aynı yazım bir kopya üretir:

ÇIKTI · C++ (KAVRAM 5)

```
$ ./H03_K05_vector_bool
vector<int>  : auto x = v[0]; x = 7;    -> x = 7, v[0] = 0
vector<bool> : auto y = v[0]; y = true; -> y = 1, v[0] = 1
vector<char> : auto z = v[0]; z = 1;    -> z = 1, v[0] = 0
```

Deney programında bu yüzden her öğesi bir bayt olan `std::vector<char>` kullanıldı: bit erişimi her okumada maskeleme gerektirir ve ölçümü dil dışı bir etkenle karıştırmak istemedik. **Kütüphane karşılıkları** algoritmayı yazmanın yerine geçmez, yazdığımızın doğruluğunu sınamanın aracıdır (Python Software Foundation, t.y.-a): permütasyon için C++ `std::next_permutation` ve Python `itertools.permutations`; alt küme ve kombinasyon için Python `itertools.product`, `itertools.combinations` ve `math.comb`.

9. Mühendis Gözüyle

Kaba kuvvet ilk doğru çözümdür ve kâhin olarak saklanır. Bu haftanın her hızlı algoritmasının yanında yavaş ama apaçık doğru bir eşi var: Örnek 5 alt küme üretimini bit maskesiyle, Örnek 8 budamalı yöntemleri kaba kuvvetle, Örnek 10 dizili denetimi taramayla, Örnek 13 Sudoku çözümünü bağımsız bir `dogrula` ile sınar. Kaba kuvveti silmeyin: bir sonraki iyileştirmenin (H13'teki dinamik programlama gibi) testi olacaktır.

Arama uzayının büyüklüğü kod yazılmadan önce hesaplanır. 8 vezir için C(64, 8) = 4 426 165 368, satır başına bir vezirle 8^8 = 16 777 216, sütun kısıtıyla 8! = 40 320: bu üç sayı, yalnızca temsil seçiminin (64 kare yerine sutun[r] dizisi) uzayı ≈ 264 kat, sütun kısıtıyla birlikte ≈ 110 000 kat küçülttüğünü, yani budamadan önce bile yaklaşık yüz binde bire indirdiğini söyler. Uygulanabilirlik duvarı (Bölüm 3.6) aynı hesabı süreye çevirir.

> MÜHENDİS GÖZÜYLE · **Ölç ve kaynakla karşılaştır: Wirth'ün 876 ve 15 720'si**
>
> Wirth (1971), sekiz vezir programının ilk çözümü bulmadan önce 876 kısmi yerleşim üretip sınadığını, 92 çözümün hepsini ise kareleri 15 720 kez sınayarak bulduğunu yazar. Wirth'ün programı sütun sütun ilerleyip satırları sırayla dener; bizimkiler satır satır ilerleyip sütunları soldan sağa dener. Bu, tahtanın köşegene göre yansımasıdır: arama ağacı aynıdır. Her denenen kare bir kısmi yerleşim olduğundan iki sayı da kare denetimi olarak sayılır.
>
> Aşağıdaki çıktıda Hata Avcısı 2'nin düzeltilmiş sürümü iki sayıyı da ölçer; Örnek 10'un dizili sürümü de N = 8'de `15 720` kare denetimi sayar. Elli yılı aşkın bir aradan sonra aynı iki sayıyı ölçmek, programımızın aynı ağacı aynı sırayla dolaştığını ve sayaçlarımızın doğru yerde olduğunu gösterir: yayımlanmış bir sonuçla karşılaştırma en güçlü testlerden biridir.

ÇIKTI

```
$ ./H03_HA2_ilk_cozumde_donmek_duzeltilmis      (ilgili satırlar)
N = 8: çözüm sayısı = 92 (beklenen 92)
tüm çözümler: ziyaret edilen düğüm = 2057, denetlenen kare = 15720
ilk çözüm: sütunlar = 0 4 7 5 2 6 1 3
ilk çözüm: ziyaret edilen düğüm = 114, denetlenen kare = 876
```

Sayaç süreden önce gelir. Bu haftanın bütün iddiaları önce bir sayaçla (çağrı, düğüm, budanan, denetim) doğrulandı, süre en son ölçüldü. Sayaçlar makineden bağımsızdır, kabul ölçütü olabilir (Mini-OBS 2 v2.2'de olduğu gibi) ve kuramla birebir karşılaştırılır; süre ise her zaman ortamıyla birlikte verilmelidir.

10. Mini-OBS 2 v2.2: AKTS Aralığına Uyan Ders Kümeleri

Bu hafta: Mini-OBS'ye bir seçmeli ders planlayıcısı ekliyoruz: AKTS toplamı belirli bir aralıkta kalan bütün ders kümelerini geri izlemeyle üretiyor, budamanın kazancını düğüm sayacıyla ve süreyle ölçüyor, sonucu bit maskesi kaba kuvvetiyle doğruluyoruz.

Problem, alt küme toplamının (Bölüm 4.3) bir aralık sürümüdür: n derslik bir havuzda AKTS toplamı [alt, üst] aralığında kalan bütün ders kümeleri. Referans çözüm **başlangıç indisli** ağacı kullanır (Örnek 7'nin kalıbı: her düğüm bir küme, 2^n düğüm), iki budama uygular (“toplam > üst ise dur” ve “kalan derslerin hepsi alınsa da alt sınıra yetmiyorsa dur”; Örnek 8'in iki sınırı) ve sonucu bit maskesi kâhiniyle (Örnek 5) sınar. Referans ölçümlere göre 30 derslik sentetik havuzda, [24, 30] aralığında üst sınır budaması 1 073 741 824 düğümlük ağacın 1 434 525'ini (%0,13) ziyaret eder ve budamasız aramadan 715 kat hızlıdır (7,2 ms'ye karşı 5,17 s); aralık ortaya kaydıkça kazanç küçülür, çünkü uygun küme sayısının kendisi üstel büyür.

Gereksinimler

- `secmeli.c`: `havuz_oku`/`havuz_yukle` (CSV `kod;ad;akts`, kurallar bölüm 5.1), `secim_yap` (budamasız, üst budamalı, üst + alt budamalı geri izleme ve bit maskesi), `maske_sozluk_karsilastir`, `maske_akts`, `maske_ders_sayisi`; ziyaret ve budanan düğüm sayaçları.
- Kümeler **sözlük sırasında** üretilmeli (geri izlemede ön sıra dolaşımdan; bit maskesinde sıralamayla).
- İsteğe bağlı en çok K ders kısıtı (K = 0: sınırsız).
- `ders_sec.c`: `--alt`, `--ust`, `--havuz`, `--en-fazla`, `--yontem`, `--sayi`, `--goster`; geçersiz kullanımda 2, havuz hatasında 1.
- `olcer.c`: en az `budamasiz`, `ust` ve `bitmaske` işlemleri (n = havuzdaki ders sayısı); CSV'de ziyaret ve süre.
- `test_obs.c` ve `test_kahin.c`: 4 derslik havuzun elle izlenmiş sonuçları; rastgele havuzlarda bit maskesi kâhini.
- Ölçüm raporu: n = 10, 12, …, ≥ 24 için düğüm sayısı ve süre tablosu, yarı logaritmik grafik, budamanın etkisinin yorumu; ölçülemeyen boyutlar için kestirim.

İpuçları

- Önce ağacı **kâğıda** çizin: 4 derslik havuzda 16 düğüm; hangi düğümler budanır?
- Durumu (toplam, adet, maske) parametreyle taşıyın; global bir dizide tutarsanız her dönüşte geri almayı unutmayın.
- Budama koşulunda `>` ile `>=` arasındaki fark bir hatadır: toplamı tam üst sınır olan küme geçerlidir.
- “Toplam aşıldı” budaması yalnızca **pozitif** değerlerde doğrudur; AKTS'yi okurken 1'den küçük değerleri reddedin.
- `1 << n` yerine `1u << n` (maske) ve `1ULL << n` (döngü sınırı) yazın; `int` ile n = 31 ve n ≥ 32 tanımsız davranıştır.
- Ölçerken kümeleri saklamayın, yalnızca sayın; yoksa ölçtüğünüz şey `realloc` olur.
- Kâhinin sonucunu karşılaştırmadan önce sözlük sırasına koyun; bit maskesinin sayısal sırası sözlük sırası değildir.

Kabul ölçütleri

1. `make` ve `make test` sıfır uyarıyla derlenir; bütün birim testleri ve `test_kahin` (300 deneme) geçer.
2. `./ders_sec --havuz testler/havuz_kucuk.csv --alt 7 --ust 9 --yontem hepsi`: 4 küme sırasıyla A+B (9), A+D (8), B+D (7), C+D (9); ziyaret: budamasız 16, üst 13 (budanan 4), üst + alt 13 (budanan 5), bit maskesi 16; her satırda "Kâhinle aynı: EVET".
3. Örnek havuz (`ornek_veri/secmeli_dersler.csv`): [18, 20] ve en fazla 4 ders → **519** küme; [30, 30] → **728**; [15, 18] → **719**; dört yöntem aynı.
4. Sınır durumları: [0, 0] → yalnızca boş küme (1); 4 derslik havuzda [19, 30] → 0 küme ve üst + alt budamada 1 ziyaret; `--alt 20 --ust 10` → çıkış 2; AKTS'si 0 olan satır → "dosya biçimi hatalı (satır 3)", çıkış 1.
5. Deterministik düğüm sayıları (`havuz_uret`, tohum 42, [24, 30]): `./olcer ust --n 20` → kontrol 22 486, ziyaret 84 472, budanan 55 568; `./olcer ust_alt --n 20` → ziyaret 84 407; `./olcer budamasiz --n 20` → ziyaret 1 048 576.
6. Budamanın etkisi (kendi makinenizde, senaryo A, n = 24): üst budamalı arama 2^{24} düğümün %2'sinden azını ziyaret eder ve budamasız aramadan en az 20 kat hızlıdır (bu makinede %1,48 ve 63 kat).
7. Budamasız arama için n iki artınca süre oranı 3,5–4,5 arasındadır (n ≥ 16).
8. `--hata-goster budama` ile kâhin hatalı budamayı yakalar, kaçırılan kümeleri basar, 1 ile çıkar.
9. `make asan` temiz.

Tablo 3.9
Mini-OBS 2 v2.2 test senaryoları (proje belgesinden)

#	Girdi	Beklenen çıktı / davranış
1	4 derslik havuz, [7, 9]	A+B, A+D, B+D, C+D; düğümler 16 / 13 / 13 / 16
2	aynı havuz, [7, 9], en fazla 1 ders	0 küme
3	aynı havuz, [0, 0]	1 küme: {} (0 AKTS, 0 ders)
4	aynı havuz, [18, 18]	1 küme: A+B+C+D
5	aynı havuz, [19, 30]	0 küme; üst + alt budamada kök budanır (1 düğüm)
6	örnek havuz, [18, 20], en fazla 4	519 küme; ilk küme BIL331 + BIL333 + BIL335 + BIL337 (19 AKTS)
7	örnek havuz, [30, 30], `--yontem hepsi --sayi`	728 küme; üst 9 901, üst + alt 7 483 düğüm
8	`--alt 20 --ust 10`	kullanım iletisi, çıkış 2
9	`testler/havuz_bozuk.csv`	"dosya biçimi hatalı (satır 3)", çıkış 1
10	`testler/havuz_tekrar_kod.csv`	"dosya biçimi hatalı (satır 4)", çıkış 1
11	`./olcer budamasiz --n 10,12,14`	ziyaret 1 024 · 4 096 · 16 384 (= 2ⁿ)
12	`./test_kahin --hata-goster budama`	"KÂHİN UYUŞMAZLIĞI", kaçırılan kümeler (toplamı tam üst sınır), çıkış 1

Öğrenciye verilen dosyalar (`secmeli.h`, örnek havuzlar, `havuz_uret`), yazılacak modüller ve iki meydan okuma görevi Mini-OBS 2 v2.2 proje belgesindedir.

11. Alıştırmalar ve Atölye

Alıştırmalar yedi türdedir: A) izleme ve çıktı tahmini, B) karmaşıklık, C) değişmez yazma, D) hata bulma, E) Parsons, F) kod yazma, G) diller arası okuma. Kodu verilen her alıştırmanın dosyası `kodlar/alistirmalar/` altındadır; önce **çalıştırmadan** tahmin edin, sonra çalıştırıp karşılaştırın. Çözümler öğretim üyesindedir.

11.1 İzleme ve çıktı tahmini (A)

Alıştırma 3.1 ★☆☆ *Çıktı tahmini*

`g(3)` çağrısının özyineleme ağacını çizin. Program ne yazdırır ve `g` toplam kaç kez çağrılır? (İpucu: yazdırma iki çağrının **arasında**.)

`alistirmalar/H03_A01_cikti_tahmini.c · satır 12–21`

```c
static void g(int n)
{
    cagri_sayisi++;
    if (n <= 0) {
        return;
    }
    g(n - 1);
    printf("%d ", n);
    g(n - 2);
}
```

Alıştırma 3.2 ★★☆ *İzleme*

Örnek 2'deki `hanoi` fonksiyonuyla (`c/H03_02_hanoi.c · satır 16–28`) `hanoi(2, 'A', 'C', 'B')` çağrısını izleyin: hangi hamleler hangi sırayla yazılır ve `hanoi` kaç kez çağrılır (n = 0 çağrıları dahil)? Çağrı sayısı için bir bağıntı yazıp n = 3 için de hesaplayın.

11.2 Karmaşıklık (B)

Alıştırma 3.3 ★☆☆ *Karmaşıklık*

$T(n) = T(n-1) + n$, $T(0) = 0$ bağıntısını yerine koyma yöntemiyle çözün ve Θ sınıfını yazın. Bu bağıntıyı doğuran bir özyinelemeli fonksiyon örneği verin.

Alıştırma 3.4 ★★☆ *Karmaşıklık*

Üçlü dizileri (her konumda 0, 1 ya da 2) üreten özyinelemeli fonksiyonun çağrı sayısı $C(n) = 3C(n-1) + 1$, $C(0) = 1$ bağıntısına uyar. Kapalı biçimini bulun. Bu makinenin C hızıyla ($\approx 1{,}7 \cdot 10^8$ düğüm/s, Bölüm 3.6) 1 saniyede en çok hangi n'ye kadar bütün diziler üretilebilir?

11.3 Değişmez yazma (C)

Alıştırma 3.5 ★★☆ *Değişmez yaz*

Örnek 6'daki `permutasyon(k)` fonksiyonu (`c/H03_06_permutasyonlar.c · satır 49–64`, Bölüm 4.2) için çağrının başında doğru olan değişmezi `yol`, `kullanildi` ve k cinsinden yazın. 60. satırdaki özyinelemeli çağrı döndükten ve 61. satır çalıştıktan **sonra** değişmezin yeniden doğru olduğunu gösterin.

Alıştırma 3.6 ★★☆ *Değişmez yaz*

Örnek 8'deki `sirali_budamali(i, toplam)` için iki budama koşulunun (`toplam + b[i] > hedef`, `toplam + kalan[i] < hedef`) neden hiçbir çözümü atmadığını, b dizisinin artan sıralı olduğu varsayımını açıkça kullanarak gösterin. b sıralı olmasaydı hangisi yanlış olurdu?

11.4 Hata bulma (D)

Alıştırma 3.7 ★★☆ *Hata bul*

n = 5, k = 3 için 10 kombinasyon beklenirken program 4 kombinasyon yazıyor. Hangi satır hatalı? Yazılan dört kümenin ortak özelliği nedir ve bu, hatayı nasıl ele verir?

alistirmalar/H03_A02_hata_bul_kombinasyon.c · satır 16–28

```
16  static void kombinasyon(int bas, int k)
17  {
18      if (uzunluk == k) {
19          sayi++;
20          printf("{%d, %d, %d} ", yol[0], yol[1], yol[2]);
21          return;
22      }
23      for (int i = bas; i < N - (k - uzunluk); i++) {
24          yol[uzunluk++] = i + 1;
25          kombinasyon(i + 1, k);
26          uzunluk--;
27      }
28  }
```

Alıştırma 3.8 ★★☆ *Hata bul*

a = {5, 4, 3, 2}, hedef 7 için iki çözüm beklenirken program `çözüm = 0` yazıyor. Hatayı bulun ve tek karakter değiştirerek düzeltin. Fonksiyonun başına bir ziyaret sayacı ekleyin: düzeltilmiş program kaç düğüm ziyaret eder (Örnek 8 ile karşılaştırın)?

alistirmalar/H03_A03_hata_bul_budama.c · satır 17–30

```c
static void budamali(int i, int toplam)
{
    if (toplam >= hedef) {
        return;                    /* "hedef aşıldı" budaması */
    }
    if (i == N) {
        if (toplam == hedef) {
            cozum_sayisi++;
        }
        return;
    }
    budamali(i + 1, toplam);
    budamali(i + 1, toplam + a[i]);
}
```

11.5 Parsons (E)

Alıştırma 3.9 ★★☆ *Parsons*

Aşağıdaki satırlar takasla permütasyon üreten fonksiyonun karıştırılmış satırlarıdır (dosya tamamlanacak; bu hâliyle derlenmez). Doğru sırayı ve girintiyi yazın; çıktı Örnek 6'nın takas sırası (123 132 213 231 321 312) olmalıdır.

alistirmalar/H03_A04_parsons_takas.c · satır 6–17

```c
        takas(&a[k], &a[i]);       /* geri al */
    if (k == n) {
static void permutasyon_takas(int k)
        permutasyon_takas(k + 1);  /* dene */
        return;
    }
{
    for (int i = k; i < n; i++) {
        perm_isle(a);
        takas(&a[k], &a[i]);       /* seç */
    }
}
```

11.6 Kod yazma (F)

Alıştırma 3.10 ★☆☆ *Kod yaz*

n bitlik ikili dizilerden **ardışık iki 1 içermeyenleri** üreten bir C programı yazın (Örnek 3'ü değiştirin: 1 yazmadan önce bir önceki bitin 0 olduğunu denetleyin; bu bir budamadır). n = 1 … 10 için dizi sayılarını yazdırın ve Fibonacci sayılarıyla ilişkisini açıklayın. C++ ve Python karşılıklarını da yazın.

Alıştırma 3.11 ★★☆ *Kod yaz*

1–9 rakamlarından **k farklı rakam** seçip toplamı s olan bütün kümeleri artan sırada yazdıran bir program yazın (başlangıç indisi + iki budama: toplam s'yi aşarsa ve kalan rakamlar yetmezse dur). k = 3, s = 15 için kaç küme vardır? Ziyaret edilen düğüm sayısını budamasız sürümle karşılaştırın.

Alıştırma 3.12 ★★★ *Kod yaz*

Örnek 10'u **simetriyle** hızlandırın: tahtanın dikey eksene göre aynası da bir çözümdür. İlk satırda yalnızca c < N/2 sütunlarını deneyip sayıyı ikiyle çarpın; N tekse orta sütunu ayrıca sayın. N = 1 … 10 için çözüm sayılarının Örnek 9'daki bilinen değerlerle aynı olduğunu denetleyin ve ziyaret edilen düğüm sayısındaki azalmayı ölçün.

11.7 Diller arası okuma (G)

Alıştırma 3.13 ★★☆ *Diller arası okuma*

Python programı ne yazdırır? İkinci `print` neden 8 değil 6 dizi içerir? Aynı “kaldığı yerden sürme” davranışını C'de nasıl elde ederdiniz?

alistirmalar/H03_A05_uretec_okuma.py · satır 8–19

```
 8  def ikili(n, on_ek=""):
 9      if n == 0:
10          yield on_ek
11          return
12      yield from ikili(n - 1, on_ek + "0")
13      yield from ikili(n - 1, on_ek + "1")
14
15
16  def main():
17      g = ikili(3)
18      print(next(g), next(g))
19      print(list(g))
```

Alıştırma 3.14 ★★☆ *Diller arası okuma*

Aşağıdaki C++ parçası ile altındaki Python parçası “aynı” işi yapıyor gibi görünüyor. Her ikisi `sonuc`'u yazdırdığında ne çıkar? Farkı Hata Avcısı 7 ile ilişkilendirin.

alistirmalar/H03_A06_kopya.cpp · satır 13–17

```
13      std::vector<std::vector<int>> sonuc;
14      std::vector<int> yol = {1, 2};
15      sonuc.push_back(yol);
16      yol.pop_back();
17      sonuc.push_back(yol);
```

Python `alistirmalar/H03_A06_kopya.py · satır 9–14`

```python
sonuc = []
yol = [1, 2]
sonuc.append(yol)
yol.pop()
sonuc.append(yol)
print(sonuc)
```

İki parçanın çıktısını önce kâğıda yazın, sonra `kodlar/alistirmalar/` altındaki iki programı çalıştırıp karşılaştırın.

11.8 Atölye 1 · Arama ağacı çizme (eşli, ≈ 15 dk)

ALGORİTMA ATÖLYESİ · **n = 3 için alt küme ve permütasyon ağaçlarını kâğıda çizin**

- Bir kişi a = {1, 2, 3} için dahil et/etme ağacını, öteki n = 3 permütasyon ağacını çizsin. Her düğüme çağrı sırasını (1, 2, …) ve o andaki `yol`'u yazın; permütasyon ağacında `kullanildi` dizisini de.
- Tahmin edin: kaç düğüm, kaç yaprak, en büyük derinlik? Sonra Örnek 4 ve Örnek 6'yı çalıştırıp çıktının sırasını kendi yapraklarınızla karşılaştırın (beklenen: 15 düğüm / 8 yaprak; 16 düğüm / 6 yaprak).
- Kâğıtları değiştirin: arkadaşınızın ağacında 6. çağrıda yığında hangi çerçeveler açık? Web sayfasındaki üretici oynatıcısıyla adım adım doğrulayın.
- Dalların sırasını değiştirirseniz (önce "dahil et") yaprakların sırası ne olur? Quiz kodu `kodlar/quiz/H03_Q01_dahil_et_once.c` ile sınayın.

11.9 Atölye 2 · N-vezir yarışı: hangi budama kaç düğüm kazandırır? (gruplar, ≈ 20 dk)

Her grup N = 8 için bir arama stratejisini çalıştırıp ölçer; sonuçlar tahtada tek bir tabloda toplanır. Ölçülecekler: ziyaret edilen düğüm sayısı, kare (ya da çift) denetimi sayısı ve süre. Bu makinedeki referans değerler aşağıdadır; sizin düğüm sayılarınız aynı, süreleriniz farklı çıkmalıdır.

Tablo 3.10
N-vezir yarışı: referans değerler (bu makine) ve doldurulacak satırlar

Strateji	Program	Düğüm (N = 8)	Denetim
Budamasız (her satırda her sütun)	deney/…/H03_deney_n_vezir budamasiz 8	19 173 961	yaprakta 28 çift
Budamalı, önceki satırları tarayarak	H03_10_n_vezir_hizli (tarama)	2 057	46 752 karşılaştırma
Budamalı, sütun/köşegen dizileriyle	H03_10_n_vezir_hizli (dizi)	2 057	15 720 kare
Yalnızca ilk çözüm	hata_avcisi/H03_HA2_…_duzeltilmis	114	876 kare
Simetri (ilk satırda sol yarı)	Alıştırma 3.12 (sizin kodunuz)	?	?
Kendi fikriniz (ör. sütunları ortadan dışa doğru denemek, ilk çözüm için)	—	?	?

Tartışma soruları: (1) Tarama ile dizi aynı sayıda düğüm ziyaret ediyor; hız farkı nereden geliyor? (2) "İlk çözüm" stratejisinin düğüm sayısı sütun deneme sırasına bağlı mıdır, "tüm çözümler"inki bağlı mıdır? (3) Simetri, kazancını ağacın hangi düzeyinde sağlar? Ölçümünüzü N = 10 ve 12 için tekrarlayın: `./H03_10_n_vezir_hizli 12 5`.

11.10 Atölye 3 · Bulmaca çözücü (eşli, ≈ 25 dk)

ALGORİTMA ATÖLYESİ · **Seçenek A — Sudoku: en az seçenekli kare önce**

Örnek 13 bu bulmacada 456 082 düğüm ziyaret eder. Sürücü kodu yazar, gözlemci tahmin eder ve sayaçları denetler; 10 dakika sonra roller değişir.

- `coz`'u değiştirin: sıradaki kare olarak **en az uygun rakamı olan** boş kareyi seçin (0 seçenekli kare varsa hemen geri dönün: bu da bir budamadır).
- Gözlemci önce tahmin etsin: düğüm sayısı kaç kat azalır? Sonra düğüm sayısını, süreyi ve `dogrula` sonucunu Örnek 13 ile karşılaştırın; düğüm başına iş arttı mı?

ALGORİTMA ATÖLYESİ · **Seçenek B — Kelime arama ızgarası**

Bir harf ızgarasında bir kelimenin, komşu (yatay, dikey, çapraz) hücrelerden geçen ve hiçbir hücreyi iki kez kullanmayan bir yol olarak bulunup bulunmadığını geri izlemeyle araştırın (Örnek 11'in şablonu: işaretle → komşuları dene → işareti kaldır).

- Girdi stdin'den: boyut, ızgara, kelimeler. Örnek ızgara `KAPI / ALEM / RTAS / TAVA`, kelimeler `KALE`, `MASA`, `TAVA` ve bulunmaması gereken `KAPAK` (ASCII harflerle başlayın).
- Budama: sıradaki harf hücredekiyle uyuşmuyorsa o yönü açmayın; ziyaret edilen hücre sayısını yazdırın. Genişletme: kelime listesini tek geçişte arayın (ortak önekler).

12. Kendini Değerlendirme

Aşağıdakilerin her birini kitaba bakmadan yapabiliyorsanız işaretleyin; yapamadıklarınız için parantez içindeki bölüme dönün.

- ☐ Bir özyinelemeli fonksiyonun özyineleme ağacını çizip çağrı sayısını ve derinliğini (yığın belleğini) ayrı ayrı belirleyebiliyorum. (3.1)
- ☐ $T(n) = T(n-1) + c$, $T(n) = 2T(n-1) + c$ ve $T(n) = n \cdot T(n-1)$ bağıntılarını yerine koyma yöntemiyle çözebiliyorum. (3.2)
- ☐ Alt küme, permütasyon ve kombinasyonları özyinelemeyle, alt kümeleri ayrıca bit maskesiyle üretebiliyor ve üretilecek nesne sayısını önceden hesaplayabiliyorum. (4.1, 4.2, 5.1, 5.2)
- ☐ Geri izleme şablonunu (seç → dene → geri al) yazabiliyor ve "geri al" adımının neden gerektiğini açıklayabiliyorum. (3.4, HA1)
- ☐ Bir budama kuralının güvenli olduğunu kısa bir kanıtla gösterebiliyor, güvenli olmadığı bir girdi bulabiliyorum. (3.5, 4.3, HA4)
- ☐ N-vezirde köşegen indislerini ($r + c$, $r - c + n - 1$) ve O(1) güvenlik denetimini açıklayabiliyorum. (4.4)
- ☐ Bir arama uzayının büyüklüğünü hesaplayıp kaba kuvvetin uygulanabilir olup olmadığına karar verebiliyorum. (3.6, 9)
- ☐ Python'daki RecursionError ile C'deki yığın taşmasının farkını ve `sonuc.append(yol)` tuzağını açıklayabiliyorum. (8, HA5, HA7)

13. İleri Bakış ve Merak Kutusu

İLERİ BAKIŞ · **Bu haftanın konusu nerede derinleşiyor?**

- H6 (böl ve fethet): özyineleme ağacı bir maliyet aracı olur; T(n) = 2T(n/2) + n gibi bağıntılarda her düzeyin işi toplanır.
- H11: labirent bir graf olarak modellenir; derinlik öncelikli arama (DFS) bu haftanın geri izlemesinin kendisidir, genişlik öncelikli arama (BFS) en kısa yolu bulur.
- H13: fib(5) ağacındaki 9 tekrar memoization ile kalkar; alt küme toplamı dinamik programlamayla, Mini-OBS'nin ders seçimi 0/1 sırt çantası olarak yeniden çözülür.
- Dal ve sınır (branch and bound) ile kısıt programlama, yöneylem araştırması ve yapay zekâ derslerinde genelleşir.

MERAK KUTUSU · **Neden C(n + 1, k) çağrı? Neden ⌊e·n!⌋?**

Kombinasyon ağacı. Örnek 7'nin ağacında d derinlikteki (d eleman seçilmiş) düğümler, n − k + d elemanlı bir önekin d'li seçimleridir; döngü sınırı yüzünden son seçilen eleman en çok n − k + d. konumdadır. Bu yüzden d derinliğinde C(n − k + d, d) düğüm vardır. d = 0 … k için toplam, "hokey sopası" özdeşliği Σ C(m + d, d) = C(m + k + 1, k) ile C(n + 1, k) eder (m = n − k). n = 10, k = 3 için 1 + 8 + 36 + 120 = 165 = C(11, 3): program çıktısındaki sayı.

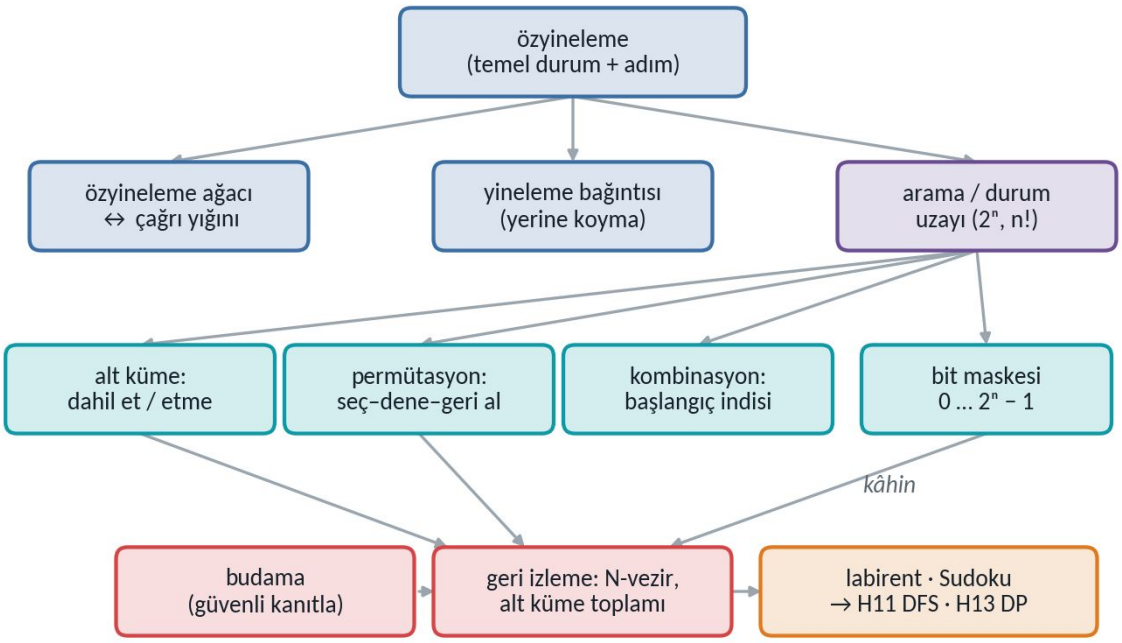
Permütasyon ağacı. k derinliğinde n·(n − 1)·…·(n − k + 1) = n!/(n − k)! düğüm vardır. Toplam n!·Σ 1/j! (j = 0 … n) eder; Σ 1/j! dizisi e'ye yakınsadığından ve kalan terimler 1/n!'den küçük kaldığından toplam tam olarak ⌊e·n!⌋'dir (n ≥ 1). n = 3: 1 + 3 + 6 + 6 = 16.

Bit maskeli N-vezir. Üç işaret dizisi üç tamsayının bitlerine dönüştürülebilir (sütunlar, sola ve sağa kayan köşegenler); bir satırdaki bütün boş sütunlar tek bir bit işlemiyle bulunur, en sağdaki 1 biti `m & -m` ile alınır. Düğüm sayısı değişmez, düğüm başına iş birkaç makine komutuna iner (Atölye 2'ye bir satır daha).

14. Özet, Kavram Haritası ve Terim Sözlüğü

- Özyineleme ağacı bütün çağrıları, çağrı yığını tek bir anı gösterir; çağrı sayısı zamanı, derinlik belleği belirler (fib(5): 15 çağrı, derinlik 5).
- Yineleme bağıntıları yerine koyma ile çözülür: T(n − 1) + c → Θ(n); 2T(n − 1) + 1 → 2^n − 1; n·T(n − 1) → n!.
- Üretme algoritmaları üç şablondan birine uyar: dahil et/etme (2^n yaprak), seç–dene–geri al (n! yaprak), başlangıç indisi (C(n, k) yaprak). Çıktı boyutu kadar iş bir alt sınırdır.
- Geri izleme, durum uzayı ağacını derinlik öncelikli dolaşır ve hiçbir çözüme götüremeyecek dalları budar. Budama güvenli olmalıdır: her kural bir kanıta dayanır.
- N-vezirde temsil (sutun[r]) arama uzayını C(64, 8)'den 8^8'e, sütun kısıtı 8!'e, köşegen budaması 2 057 düğüme indirir; dizilerle denetim O(1)'dir.
- Uygulanabilirlik duvarı: bu makinede 1 saniyede C ile 2^n için n ≈ 27, n! için n ≈ 11. Budamanın kazancı dil farkından çok büyüktür.
- Dil farkları: Python'da derinlik sınırı (RecursionError), C'de yığın taşması (sessiz, Linux'ta tipik olarak SIGSEGV); Python `append(yol)` başvuru ekler, C++ `push_back(yol)` kopyalar.

Şekil 3.15
Hafta 3 kavram haritası



Tablo 3.11
Terim sözlüğü (Yol Haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Not
özyineleme; temel durum; özyinelemeli adım	recursion; base case; recursive step	AP-1 H9
özyineleme ağacı; çağrı yığını; yığın çerçevesi	recursion tree; call stack; stack frame	"yığın" her zaman stack
yineleme bağıntısı; yerine koyma yöntemi	recurrence; substitution (iteration) method	
kaba kuvvet; arama uzayı; durum uzayı ağacı	brute force (exhaustive search); search space; state-space tree	
geri izleme; budama	backtracking; pruning	"geri izleme" yalnızca backtracking
alt küme; permütasyon; kombinasyon; sözlük sırası	subset; permutation; combination; lexicographic order	
bit maskesi; üreteç	bitmask; generator	Python `yield`
kâhin; karşılaştırmalı test	test oracle; differential testing	
yığın taşması; kuyruk özyineleme	stack overflow; tail recursion	
tanımsız davranış; öbek	undefined behavior; heap	öbek: dinamik bellek bölgesi
kısıt yayılımı; en az seçenekli kare önce	constraint propagation; minimum remaining values (MRV)	Sudoku, Atölye 3
dal ve sınır	branch and bound	İleri Bakış

15. Kaynaklar

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Dahl, O.-J., Dijkstra, E. W., & Hoare, C. A. R. (1972). *Structured programming*. Academic Press.

ISO/IEC. (2017). *Programming languages — C++* (ISO/IEC Standard No. 14882:2017). International Organization for Standardization.

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Levitin, A. (2012). *Introduction to the design and analysis of algorithms* (3rd ed.). Pearson.

Python Software Foundation. (t.y.-a). *The Python standard library*. Erişim tarihi 12 Ekim 2026, https://docs.python.org/3/library/

Python Software Foundation. (t.y.-b). *What's new in Python 3.11*. Erişim tarihi 13 Ekim 2026, https://docs.python.org/3/whatsnew/3.11.html

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley.

Skiena, S. S. (2020). *The algorithm design manual* (3rd ed.). Springer.

Tshukudu, E., & Cutts, Q. (2020). Understanding conceptual transfer for students learning new programming languages. In *Proceedings of the 2020 ACM Conference on International Computing Education Research* (pp. 227–237). ACM. https://doi.org/10.1145/3372782.3406270

Wirth, N. (1971). Program development by stepwise refinement. *Communications of the ACM*, *14*(4), 221–227. https://doi.org/10.1145/362575.362577

Ek A. Dil Karşılıkları

Bu ekte örnek merdiveninin on üç basamağının C++ ve Python sürümlerinden algoritmanın çekirdeğini oluşturan fonksiyonlar, dosyadaki satır numaralarıyla verilir; tam dosyalar `kodlar/cpp/` ve `kodlar/python/` altındadır. Hardal satırlar C sürümünden **anlam olarak** farklılaşan satırlardır (`kodlar/ornekler.json` → `farklar`); fonksiyon dışındaki farklar not kutusunda açıklanır. Fonksiyon ve değişken adları üç dilde aynıdır. Amiral örneklerin (4, 6, 8, 10) çekirdek fonksiyonlarının Python sürümü ve C++'ın farklılaşan satırları Bölüm 4'tedir (C++ çekirdek fonksiyonları C'ninkilerle satır satır aynı yapıdadır); burada aynı dosyaların öteki fonksiyonları gösterilir.

Tablo 3.12
Derleme ve çalıştırma komutları (X: örneğin kök adı, ör. H03_04_alt_kumeler)

Dil	Derleme / çalıştırma	Ek dosya, girdi ya da bayrak
C	`gcc -std=c17 -Wall -Wextra -pedantic -O2 X.c -o X · ./X`	Örnek 8: + ortak_rastgele.c; Örnek 10, 13: + zamanlayici.c
C++	`g++ -std=c++17 -Wall -Wextra -pedantic -O2 X.cpp -o X · ./X`	ortak başlıklar (.hpp) yalnızca #include
Python	`python3 X.py (Windows: py X.py)`	Python 3.10 ve üstü; yalnızca standart kütüphane
Hepsi	`Örnek 11 ve 13 girdiyi stdin'den okur`	./X < ../girdiler/X.txt
Hepsi	`Örnek 12 bilerek -O0 ile derlenir`	-O2'de GCC özyinelemeyi döngüye çevirir

A.1 Örnek 1 · Özyinelemeli toplam ve Fibonacci'de çağrı sayaçları; fib(5) çağrı ağacı

C++ **cpp/H03_01_ozyineleme_sayac.cpp · satır 39–49**

```cpp
static long long fib(int n, int d)
{
    cagri_sayisi++;
    if (d > en_buyuk_derinlik) {
        en_buyuk_derinlik = d;
    }
    if (n < 2) {
        return n;                       // temel durum
    }
    return fib(n - 1, d + 1) + fib(n - 2, d + 1);
}
```

Python **python/H03_01_ozyineleme_sayac.py · satır 31–39**

```python
def fib(n, d):
    """n. Fibonacci sayısı: F(0) = 0, F(1) = 1"""
    global cagri_sayisi, en_buyuk_derinlik
    cagri_sayisi += 1
    if d > en_buyuk_derinlik:
        en_buyuk_derinlik = d
    if n < 2:
        return n                        # temel durum
    return fib(n - 1, d + 1) + fib(n - 2, d + 1)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Python'da modül düzeyindeki sayaçları bir fonksiyonun içinde değiştirmek için `global` bildirimi gerekir; C ve C++'ta dosya kapsamlı `static` değişkenler doğrudan güncellenir. Girinti C'de döngüyle, C++'ta `std::string(2 * (d - 1), ' ')` ile, Python'da `"  " * (d - 1)` dizgi tekrarıyla üretilir. (Fonksiyon dışındaki fark satırları: C++ 17, 58; Python 22, 48, 62.)

A.2 Örnek 2 · Hanoi Kuleleri: T(n) = 2T(n−1) + 1 bağıntısının hamle sayacıyla doğrulanması

C++ **cpp/H03_02_hanoi.cpp · satır 17–29**

```cpp
static void hanoi(int n, char kaynak, char hedef, char yardimci)
{
    if (n == 0) {
        return;                                 // temel durum: disk yok
    }
    hanoi(n - 1, kaynak, yardimci, hedef);   // üstteki n-1 disk
    hamle_sayisi++;                             // en büyük disk
    if (listele) {
        std::cout << std::setw(2) << hamle_sayisi << ". hamle: disk " << n
                  << "  " << kaynak << " -> " << hedef << "\n";
    }
    hanoi(n - 1, yardimci, hedef, kaynak);   // n-1 disk yeniden
}
```

Python **python/H03_02_hanoi.py · satır 13–22**

```python
def hanoi(n, kaynak, hedef, yardimci):
    """n diski kaynak çubuğundan hedef çubuğuna taşır"""
    global hamle_sayisi
    if n == 0:
        return                                  # temel durum: disk yok
    hanoi(n - 1, kaynak, yardimci, hedef)    # üstteki n-1 disk
    hamle_sayisi += 1                           # en büyük disk
    if listele:
        print(f"{hamle_sayisi:2d}. hamle: disk {n}  {kaynak} -> {hedef}")
    hanoi(n - 1, yardimci, hedef, kaynak)    # n-1 disk yeniden
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'ta `1 << n` bir `int` ifadesidir ve n ≥ 31'de taşar; bu yüzden `1LL << n` yazılır. Python'da tamsayılar sınırsızdır, `1 << n` her n için doğrudur. Çubuk adları C/C++'ta `char` ('A'), Python'da tek karakterlik dizgidir ("A"). (Fonksiyon dışındaki fark satırları: C++ 59; Python 29, 51.)

A.3 Örnek 3 · n bitlik tüm ikili diziler: dahil et/etme ağacı

C++ cpp/H03_03_ikili_diziler.cpp · satır 20–34

```cpp
static void ikili_uret(int k, int n)
{
    cagri_sayisi++;
    if (k == n) {                   // yaprak: dizi tamamlandı
        dizi_sayisi++;
        if (yazdir_mi) {
            std::cout << dizi << "\n";
        }
        return;
    }
    dizi[k] = '0';                  // sol dal: k. bit 0
    ikili_uret(k + 1, n);
    dizi[k] = '1';                  // sağ dal: k. bit 1
    ikili_uret(k + 1, n);
}
```

Python python/H03_03_ikili_diziler.py · satır 15–27

```python
def ikili_uret(k, n):
    """dizi[0..k-1] belirlendi; dizi[k..n-1] için iki seçeneği dene"""
    global cagri_sayisi, dizi_sayisi
    cagri_sayisi += 1
    if k == n:                      # yaprak: dizi tamamlandı
        dizi_sayisi += 1
        if yazdir_mi:
            print("".join(dizi))
        return
    dizi[k] = "0"                   # sol dal: k. bit 0
    ikili_uret(k + 1, n)
    dizi[k] = "1"                   # sağ dal: k. bit 1
    ikili_uret(k + 1, n)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C'de dizi bir `char` dizisidir ve yazdırmadan önce `'\0'` ile sonlandırılır; C++'ta `std::string` kendi uzunluğunu bilir. Python dizgileri değiştirilemez (immutable) olduğundan karakter listesi kullanılır ve `"".join` ile birleştirilir. (Fonksiyon dışındaki fark satırları: C++ 14, 39; Python 9.)

A.4 Örnek 4 · Bir kümenin tüm alt kümeleri (özyinelemeli üretme şablonu)

C++ cpp/H03_04_alt_kumeler.cpp · satır 23–34

```cpp
static void kume_isle()
{
    kume_sayisi++;
    eleman_toplami += static_cast<long long>(yol.size());
    if (yazdir_mi) {
        std::cout << std::setw(2) << kume_sayisi << ": {";
        for (std::size_t j = 0; j < yol.size(); j++) {
            std::cout << (j > 0 ? ", " : "") << yol[j];
        }
        std::cout << "}\n";
    }
}
```

Python python/H03_04_alt_kumeler.py · satır 18–27

```python
def kume_isle():
    """Bir alt küme tamamlandı: say ve (istenirse) yaz"""
    global kume_sayisi, eleman_toplami
    kume_sayisi += 1
    eleman_toplami += len(yol)
    if yazdir_mi:
        print(f"{kume_sayisi:2d}: {{", end="")
        for j in range(len(yol)):
            print((", " if j > 0 else "") + str(yol[j]), end="")
        print("}")
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> C'de seçilenler sabit dizi ve `uzunluk` sayacıyla tutulur, geri alma `uzunluk--` ile yapılır; C++'ta `push_back`/`pop_back`, Python'da `append`/`pop` aynı yığın davranışını verir. Geri alma adımı üç dilde de özyinelemeli çağrının hemen ardındadır. (Fonksiyon dışındaki fark satırları: C++ 45, 47; Python 38, 40.)

A.5 Örnek 5 · Aynı problem bit maskesiyle (0 … $2^n - 1$); Örnek 4 ile aynı küme ailesi ve sıra

C++ cpp/H03_05_bit_maskesi.cpp · satır 43–55

```cpp
static void maske_ile_uret(int n)
{
    std::cout << "maske  ikili  a[0] soldaki bit  a[0] sağdaki bit\n";
    for (unsigned maske = 0; maske < (1u << n); maske++) {
        std::cout << std::setw(5) << maske << "  ";
        for (int b = n - 1; b >= 0; b--) {
            std::cout << (((maske >> b) & 1u) ? '1' : '0');
        }
        std::cout << "    " << std::left << std::setw(16)
                  << kume_metni(maske, n, true) << std::right << "  "
                  << kume_metni(maske, n, false) << "\n";
    }
}
```

Python **python/H03_05_bit_maskesi.py · satır 33–41**

```python
def maske_ile_uret(n):
    """Bit maskesiyle üretim: her maske bir alt küme"""
    print("maske  ikili  a[0] soldaki bit  a[0] sağdaki bit")
    for maske in range(1 << n):
        print(f"{maske:5d}  ", end="")
        for b in range(n - 1, -1, -1):
            print("1" if (maske >> b) & 1 else "0", end="")
        print(f"    {kume_metni(maske, n, True):<16}  "
              f"{kume_metni(maske, n, False)}")
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> C ve C++'ta maske `unsigned` türündedir ve `1u << n` ile kurulur (işaretli `1 << n`, n ≥ 31'de taşar: Hata Avcısı 6). Python'da `range(1 << n)` her n için doğrudur, ama döngü yine 2^n adım sürer. (Fonksiyon dışındaki fark satırları: C++ 18, 63; Python 48, 50.)

A.6 Örnek 6 · Tüm permütasyonlar — kullanildi dizisiyle, sözlük sırasında; takasla üretim

C++ **cpp/H03_06_permutasyonlar.cpp · satır 66–78**

```cpp
static void permutasyon_takas(int k)
{
    cagri_sayisi++;
    if (k == n) {
        perm_isle(a);
        return;
    }
    for (int i = k; i < n; i++) {
        std::swap(a[k], a[i]);     // seç
        permutasyon_takas(k + 1);  // dene
        std::swap(a[k], a[i]);     // geri al
    }
}
```

Python **python/H03_06_permutasyonlar.py · satır 52–62**

```python
def permutasyon_takas(k):
    """Takasla üretim: a[k..n-1] içinden her elemanı k. konuma getir"""
    global cagri_sayisi
    cagri_sayisi += 1
    if k == n:
        perm_isle(a)
        return
    for i in range(k, n):
        a[k], a[i] = a[i], a[k]     # seç
        permutasyon_takas(k + 1)    # dene
        a[k], a[i] = a[i], a[k]     # geri al
```

> DİL PENCERESİ · **Farklılaşan satırlar**
>
> C++'ta `onceki = p` vektörü kopyalar; Python'da aynı yazım yalnızca aynı listeye ikinci bir ad verir, bu yüzden `p[:]` ile kopya alınır (benzer sözdizimi, farklı anlam). Takas C'de yardımcı fonksiyonla, C++'ta `std::swap`, Python'da çoklu atamayla yapılır. (Fonksiyon dışındaki fark satırları: C++ 18, 38; Python 30.)

A.7 Örnek 7 · k-kombinasyonlar: başlangıç indisiyle tekrarsız üretim ve C(n, k) sayımı

C++ **cpp/H03_07_kombinasyonlar.cpp · satır 34–58**

```cpp
static void kombinasyon(int bas, int k)
{
    cagri_sayisi++;
    int uzunluk = static_cast<int>(yol.size());
    if (uzunluk == k) {            // k eleman seçildi
        komb_isle();
        return;
    }
    // i en çok n - (k - uzunluk): sağda yeterli eleman kalmalı
    for (int i = bas; i <= n - (k - uzunluk); i++) {
        yol.push_back(a[i]);       // seç
        kombinasyon(i + 1, k);     // dene: yalnızca i'den sonrakiler
        yol.pop_back();            // geri al
    }
}

// C(m, k) = m! / (k! (m-k)!), çarpımsal formülle (her adım tam bölünür)
static long long kombinasyon_sayisi(int m, int k)
{
    long long c = 1;
    for (int i = 1; i <= k; i++) {
        c = c * (m - k + i) / i;   // c = C(m-k+i, i)
    }
    return c;
}
```

Python **python/H03_07_kombinasyonlar.py · satır 27–47**

```python
def kombinasyon(bas, k):
    """a[bas..n-1] içinden k - len(yol) eleman daha seç"""
    global cagri_sayisi
    cagri_sayisi += 1
    uzunluk = len(yol)
    if uzunluk == k:               # k eleman seçildi
        komb_isle()
        return
    # i en çok n - (k - uzunluk): sağda yeterli eleman kalmalı
    for i in range(bas, n - (k - uzunluk) + 1):
        yol.append(a[i])           # seç
        kombinasyon(i + 1, k)      # dene: yalnızca i'den sonrakiler
        yol.pop()                  # geri al


```

```python
def kombinasyon_sayisi(m, k):
    """C(m, k) = m! / (k! (m-k)!), çarpımsal formülle (her adım tam bölünür)"""
    c = 1
    for i in range(1, k + 1):
        c = c * (m - k + i) // i    # c = C(m-k+i, i); // tamsayı bölmesi
    return c
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Python'da `range` üst sınırı dahil etmez: C'deki `i <= n - (k - uzunluk)` koşulu `range(bas, n - (k - uzunluk) + 1)` olur. C(n, k) hesabında Python'da `//` gerekir; `/` ondalıklı (float) sonuç verir. Seçilenler C'de dizi ve `uzunluk` sayacıyla, C++'ta `push_back`/`pop_back`, Python'da `append`/`pop` ile tutulur.

A.8 Örnek 8 · Alt küme toplamı: kaba kuvvet ve budamalı geri izleme (ziyaret edilen düğüm)

C++ `cpp/H03_08_alt_kume_toplami.cpp · satır 41–54`

```cpp
static void kaba_kuvvet(int i, long long toplam)
{
    ziyaret++;
    if (i == n) {
        if (toplam == hedef) {
            cozum_isle();
        }
        return;
    }
    kaba_kuvvet(i + 1, toplam);             // a[i] alınmaz
    yol.push_back(a[i]);                    // seç
    kaba_kuvvet(i + 1, toplam + a[i]);      // a[i] alınır
    yol.pop_back();                         // geri al
}
```

Python `python/H03_08_alt_kume_toplami.py · satır 30–41`

```python
def kaba_kuvvet(i, toplam):
    """1) Kaba kuvvet: ağacın tamamı; yalnızca yaprakta denetim"""
    global ziyaret
    ziyaret += 1
    if i == n:
        if toplam == hedef:
            cozum_isle()
        return
    kaba_kuvvet(i + 1, toplam)              # a[i] alınmaz
    yol.append(a[i])                        # seç
    kaba_kuvvet(i + 1, toplam + a[i])       # a[i] alınır
    yol.pop()                               # geri al
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Ortak LCG üç dilde aynı diziyi üretir: C'de `rastgele_dizi(a, n, 1, 40)` var olan diziyi doldurur, C++ ve Python'da yeni bir vektör/liste döndürür. Budama koşulu `toplam > hedef` üç dilde aynıdır ve yalnızca negatif olmayan sayılarda geçerlidir (Hata Avcısı 4). Seçim yığını C'de `yol[uzunluk++]` … `uzunluk--`, C++'ta `push_back`/`pop_back`, Python'da `append`/`pop` ile tutulur. (Fonksiyon dışındaki fark satırları: C++ 71, 73, 156; Python 56, 58, 129, 137.)

A.9 Örnek 9 · N-vezir: geri izleme ve N = 1…10 için çözüm sayıları (N = 8 için 92)

C++ **cpp/H03_09_n_vezir.cpp · satır 43–74**

```cpp
static void yerlestir(int r)
{
    dugum_sayisi++;
    if (r == n) {                  // n vezir yerleşti: çözüm
        cozum_sayisi++;
        if (cozum_sayisi <= yazilacak) {
            std::cout << "çözüm " << cozum_sayisi << ": sütunlar =";
            for (int i = 0; i < n; i++) {
                std::cout << " " << sutun[i];
            }
            std::cout << "\n";
            tahta_yaz();
        }
        return;
    }
    for (int c = 0; c < n; c++) {
        if (guvenli_mi(r, c)) {
            sutun[r] = c;          // seç
            yerlestir(r + 1);      // dene
        }                          // geri al: sutun[r] yeniden yazılır
    }
}

static void coz(int boyut, int yaz)
{
    n = boyut;
    sutun.assign(n, 0);
    yazilacak = yaz;
    cozum_sayisi = 0;
    dugum_sayisi = 0;
    yerlestir(0);
}
```

Python **python/H03_09_n_vezir.py · satır 33–60**

```python
def yerlestir(r):
    """0..r-1 satırlarına vezir yerleşti; r. satır için sütun ara"""
    global cozum_sayisi, dugum_sayisi
    dugum_sayisi += 1
    if r == n:                      # n vezir yerleşti: çözüm
        cozum_sayisi += 1
        if cozum_sayisi <= yazilacak:
```

```
            print(f"çözüm {cozum_sayisi}: sütunlar =", end="")
            for i in range(n):
                print(f" {sutun[i]}", end="")
            print()
            tahta_yaz()
        return
    for c in range(n):
        if guvenli_mi(r, c):
            sutun[r] = c            # seç
            yerlestir(r + 1)        # dene
        # geri al: sutun[r] bir sonraki c ile yeniden yazılır


def coz(boyut, yaz):
    global n, sutun, yazilacak, cozum_sayisi, dugum_sayisi
    n = boyut
    sutun = [0] * n
    yazilacak = yaz
    cozum_sayisi = 0
    dugum_sayisi = 0
    yerlestir(0)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Tahta n×n matris yerine tek boyutlu `sutun[r]` dizisiyle tutulur. Python'da `coz` içinde `sutun = [0] * n` ile adı yeni bir listeye bağlamak `global` gerektirir; C++'ta `sutun.assign(n, 0)` aynı vektörü yeniden boyutlandırır. (Fonksiyon dışındaki fark satırları: C++ 15.)

A.10 Örnek 10 · N-vezir: sütun/köşegen dizileriyle O(1) güvenlik denetimi ve süre karşılaştırması

C++ **cpp/H03_10_n_vezir_hizli.cpp · satır 42–55**

```
static void yerlestir_tarama(int r)
{
    dugum_sayisi++;
    if (r == n) {
        cozum_sayisi++;
        return;
    }
    for (int c = 0; c < n; c++) {
        if (guvenli_mi_tarama(r, c)) {
            sutun[r] = c;
            yerlestir_tarama(r + 1);
        }
    }
}
```

Python **python/H03_10_n_vezir_hizli.py · satır 36–45**

```python
def yerlestir_tarama(r):
    global dugum_sayisi, cozum_sayisi
    dugum_sayisi += 1
    if r == n:
        cozum_sayisi += 1
        return
    for c in range(n):
        if guvenli_mi_tarama(r, c):
            sutun[r] = c
            yerlestir_tarama(r + 1)
```

> **DİL PENCERESİ · Farklılaşan satırlar**
>
> Ölçülecek iş C'de fonksiyon işaretçisi ve `void *` veriyle, C++'ta lambda (`[&] { … }`), Python'da `lambda:` ile `olc_ortanca`'ya verilir. C++'taki `std::vector<bool>` her öğeyi bir bitte saklayan özel bir kaptır. Süreler stderr'e yazıldığından stdout üç dilde aynıdır. (Fonksiyon dışındaki fark satırları: C++ 22, 117, 140; Python 104, 123.)

A.11 Örnek 11 · Labirentte yol bulma: ızgarada geri izleme ve yolun işaretlenmesi

C++ **cpp/H03_11_labirent.cpp · satır 26–53**

```cpp
static bool gecilebilir(int r, int c)
{
    return r >= 0 && r < satir_sayisi && c >= 0 && c < sutun_sayisi
           && (izgara[r][c] == '.' || izgara[r][c] == 'E');
}

// (r, c)'den çıkışa yol ara; bulunursa true döner
static bool yol_bul(int r, int c)
{
    ziyaret++;
    if (izgara[r][c] == 'E') {
        return true;                    // çıkışa ulaşıldı
    }
    if (izgara[r][c] != 'S') {
        izgara[r][c] = '*';             // seç: hücre yola eklenir
    }
    for (int d = 0; d < 4; d++) {
        int yr = r + DR[d], yc = c + DC[d];
        if (gecilebilir(yr, yc) && yol_bul(yr, yc)) {
            return true;                // yol bulundu: '*'lar kalır
        }
    }
    if (izgara[r][c] != 'S') {
        izgara[r][c] = 'x';             // geri al: buradan çıkış yok
    }
    geri_donulen++;
    return false;
}
```

Python `python/H03_11_labirent.py · satır 23–44`

```python
def gecilebilir(r, c):
    """(r, c) ızgarada ve henüz denenmemiş bir boş hücre ya da çıkış mı?"""
    return (0 <= r < satir_sayisi and 0 <= c < sutun_sayisi
            and izgara[r][c] in (".", "E"))


def yol_bul(r, c):
    """(r, c)'den çıkışa yol ara; bulunursa True döner"""
    global ziyaret, geri_donulen
    ziyaret += 1
    if izgara[r][c] == "E":
        return True                    # çıkışa ulaşıldı
    if izgara[r][c] != "S":
        izgara[r][c] = "*"             # seç: hücre yola eklenir
    for d in range(4):
        yr, yc = r + DR[d], c + DC[d]
        if gecilebilir(yr, yc) and yol_bul(yr, yc):
            return True                # yol bulundu: '*'lar kalır
    if izgara[r][c] != "S":
        izgara[r][c] = "x"             # geri al: buradan çıkış yok
    geri_donulen += 1
    return False
```

DİL PENCERESİ · **Farklılaşan satırlar**

Python dizgileri değiştirilemez; ızgaranın her satırı karakter listesine çevrilir (`list(...)`) ve yazdırılırken `"".join` ile birleştirilir. `0 <= r < satir_sayisi` zincir karşılaştırması C'deki iki koşulun kısaltmasıdır. C++'ta `std::string` öğeleri yerinde değiştirilebilir. (Fonksiyon dışındaki fark satırları: C++ 18, 62; Python 64, 103.)

A.12 Örnek 12 · Özyineleme sınırı deneyi: Python RecursionError ve C'de yığın taşması

C++ `cpp/H03_12_ozyineleme_siniri.cpp · satır 17–58`

```cpp
static long long toplam(long long n)
{
    if (n == 0) {
        return 0;
    }
    return n + toplam(n - 1);
}

int main(int argc, char* argv[])
{
    if (argc > 1) {                    // isteğe bağlı: tek derinlik
        long long n = -1;
        try {
            std::size_t son = 0;
            n = std::stoll(argv[1], &son);
            if (son != std::string(argv[1]).size()) {
                n = -1;
```

```
            }
        } catch (const std::exception&) {
            n = -1;
        }
        if (n < 0) {
            std::cerr << "kullanım: " << argv[0] << " [n >= 0]\n";
            return 2;
        }
        std::cout << "toplam(" << n << ") = " << toplam(n) << "\n";
        return 0;
    }
    const long long derinlikler[] = {900, 1000, 10000};
    std::cout << "Özyineleme sınırı deneyi — C++\n";
    std::cout << "toplam(n) = n + toplam(n - 1), toplam(0) = 0\n";
    std::cout << "dilin koyduğu derinlik sınırı: yok\n";
    for (long long n : derinlikler) {
        std::cout << "derinlik " << std::setw(6) << n << ": toplam = "
                  << toplam(n) << "\n";
    }
    std::cout << "C++'ta sınır iş parçacığının yığın boyutudur (Linux'ta "
                 "tipik\nolarak 8 MB; ulimit -s). Aşılırsa dil uyarmaz; "
                 "Linux'ta süreç\ngenellikle SIGSEGV (Segmentation fault) "
                 "ile sonlanır.\n";
    return 0;
}
```

Python **python/H03_12_ozyineleme_siniri.py · satır 15–44**

```
def toplam(n):
    if n == 0:
        return 0
    return n + toplam(n - 1)


def main():
    if len(sys.argv) > 1:               # isteğe bağlı: tek derinlik
        try:
            n = int(sys.argv[1])
        except ValueError:
            n = -1
        if n < 0:
            print(f"kullanım: {sys.argv[0]} [n >= 0]", file=sys.stderr)
            sys.exit(2)
        sys.setrecursionlimit(n + 100)  # sınırı bilerek yükselt
        print(f"toplam({n}) = {toplam(n)}")
        return
    derinlikler = [900, 1000, 10000]
    print("Özyineleme sınırı deneyi — Python")
    print("toplam(n) = n + toplam(n - 1), toplam(0) = 0")
    print(f"sys.getrecursionlimit() = {sys.getrecursionlimit()}")
    for n in derinlikler:
        try:
            print(f"derinlik {n:6d}: toplam = {toplam(n)}")
        except RecursionError as hata:
            print(f"derinlik {n:6d}: RecursionError yakalandı")
```

```
            print(f"  ileti: {hata}", file=sys.stderr)
    print("Python'da sınır yorumlayıcının koruma sayacıdır; aşılınca\n"
          "RecursionError fırlatılır ve program yakalayıp sürdürebilir.")
```

DİL PENCERESİ · Δ — dile özgü çıktı

Python, `sys.getrecursionlimit()` (varsayılan 1000) kadar çerçeveyi aşan çağrıda `RecursionError` fırlatır ve program bunu yakalayıp sürdürebilir; C/C++'ta dilin bir sınırı yoktur, sınır yığın boyutudur (Linux'ta tipik 8 MB) ve aşıldığında süreç uyarısız çöker. `-O2` ile GCC bu özyinelemeyi döngüye çevirdiği için örnek `-O0` ile derlenir.

ÇIKTI · C++

```
$ g++ -std=c++17 -Wall -Wextra -pedantic -O0 H03_12_ozyineleme_siniri.cpp -o
H03_12_ozyineleme_siniri $ ./H03_12_ozyineleme_siniri
Özyineleme sınırı deneyi — C++
toplam(n) = n + toplam(n - 1), toplam(0) = 0
dilin koyduğu derinlik sınırı: yok
derinlik    900: toplam = 405450
derinlik   1000: toplam = 500500
derinlik  10000: toplam = 50005000
C++'ta sınır iş parçacığının yığın boyutudur (Linux'ta tipik
olarak 8 MB; ulimit -s). Aşılırsa dil uyarmaz; Linux'ta süreç
genellikle SIGSEGV (Segmentation fault) ile sonlanır.
```

C çıktısı Bölüm 8'dedir; C++ çıktısı C'ninkiyle yalnızca başlık satırında ve “C++'ta” sözcüğünde ayrışır, Python çıktısında ise 1000 ve 10 000 derinlikleri RecursionError verir.

A.13 Örnek 13 · Sudoku çözücüsü (meydan okuma): kısıt yayılımı olmadan geri izleme

C++ cpp/H03_13_sudoku.cpp · satır 48–68

```cpp
static bool coz(int k)
{
    dugum_sayisi++;
    while (k < 81 && tablo[k / 9][k % 9] != 0) {
        k++;                          // dolu kareleri atla
    }
    if (k == 81) {
        return true;                  // boş kare kalmadı: çözüldü
    }
    int r = k / 9, c = k % 9;
    for (int d = 1; d <= 9; d++) {
        if (uygun_mu(r, c, d)) {
            koy(r, c, d, true);       // seç
            if (coz(k + 1)) {         // dene
                return true;          // tek çözüm yeter: dur
            }
            koy(r, c, d, false);      // geri al
        }
    }
    return false;                     // hiçbir rakam olmadı: geri dön
```

```
68 }
```

Python **python/H03_13_sudoku.py · satır 38–54**

```python
def coz(k):
    """k = 0..80 (satır öncelikli kare no); k'dan sonraki ilk boş kareyi
    doldur"""
    global dugum_sayisi
    dugum_sayisi += 1
    while k < 81 and tablo[k // 9][k % 9] != 0:
        k += 1                      # dolu kareleri atla
    if k == 81:
        return True                 # boş kare kalmadı: çözüldü
    r, c = k // 9, k % 9
    for d in range(1, 10):
        if uygun_mu(r, c, d):
            koy(r, c, d, True)      # seç
            if coz(k + 1):          # dene
                return True         # tek çözüm yeter: dur
            koy(r, c, d, False)     # geri al
    return False                    # hiçbir rakam olmadı: geri dön
```

DİL PENCERESİ · **Farklılaşan satırlar**

Python'da 9×9 tablo `[[0] * 9 for _ in range(9)]` ile kurulur; `[[0] * 9] * 9` aynı satır listesini dokuz kez paylaştırırdı. C++'ta `std::array` sabit boyutlu ve değer anlamlıdır; `{}` ile sıfırlanır. (Fonksiyon dışındaki fark satırları: C++ 17, 123; Python 13, 68.)